

USENIX

THE ADVANCED COMPUTING
SYSTEMS ASSOCIATION

GraphAce: Secure Two-Party Graph Analysis Achieving Communication Efficiency

Jiping Yu, *Tsinghua University and Ant Group;* Kun Chen, *Ant Group;*
Yunyi Chen and Xiaoyu Fan, *Tsinghua University and Ant Group;* Xiaowei Zhu
and Cheng Hong, *Ant Group;* Wenguang Chen, *Tsinghua University and Ant Group*

https://www.usenix.org/conference/usenixsecurity25/presentation/yu-jiping

This paper is included in the Proceedings of the
34th USENIX Security Symposium.

August 13–15, 2025 • Seattle, WA, USA

978-1-939133-52-6

Open access to the Proceedings of the
34th USENIX Security Symposium is sponsored by USENIX.

ARTIFACT EVALUATED
usenix ASSOCIATION
AVAILABLE

GraphAce: Secure Two-Party Graph Analysis Achieving Communication Efficiency

Jiping Yu [1,2,*], Kun Chen [2], Yunyi Chen [1,2,*], Xiaoyu Fan [1,2,*], Xiaowei Zhu [2], Cheng Hong [2], and Wenguang Chen [1,2]

[1] *Tsinghua University*
[2] *Ant Group*

Abstract

Graph analysis has achieved success in challenging tasks such as importance measures, fraud detection, and anti-money laundering. For a deep understanding of various complex systems in real life, a whole graph may involve data and connections from multiple sources. Secure multi-party computation is suitable for this scenario, which allows untrusted parties to compute collectively without revealing their individual data. However, a significant challenge is the high communication overhead, especially during intricate computations of large-scale input, and existing secure graph analysis frameworks incur a lower bound of $\Omega(|V|+|E|)$ communication per iteration, where V and E denote vertices and edges, respectively.

This paper proposes GraphAce, an efficient secure two-party graph analysis framework, which adopts a distinct technical roadmap from existing solutions. We identify and address the security challenges when utilizing local graph data of parties, with the mixed primitives system of homomorphic encryption and secret sharing, and a novel ChaosTable data structure that protects privacy during cross-party computation. Consequently, GraphAce eliminates any network traffic related to the edges. For each iteration, it achieves low complexities of $\Theta(|V|)$ communication, breaking the $\Omega(|V|+|E|)$ lower bound of previous secure solutions, and $\Theta(|V|+|E|)$ computation, which is the same as insecure methods.

Evaluations show that GraphAce exceeds previous methods by up to tens of thousands of times in speed and saves up to 99.99% communication, depending on the application and the network. This is the first secure two-party graph analysis framework capable of processing over 1 million vertices and 132 million edges in a reasonable time, which is 128× larger than previous reports, to the best of our knowledge.

1 Introduction

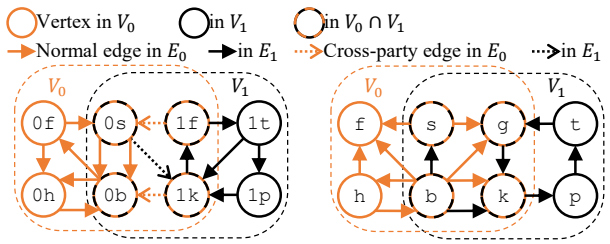
Graph analysis has shown its effects in many areas of the real world, such as website scoring [45], fraud detection [48], and protein classification based on community detection [36]. In reality, since the data are often distributed over multiple entities (e.g. each bank owns its transaction records), a natural idea is to gather all data and perform an analysis on the whole graph. Thus, the analysis results can be of higher quality when input data from multiple entities are incorporated instead of relying solely on the data from each individual entity.

However, in numerous instances, graph data from various entities, such as transaction records from distinct banks, are not mutually shared, possibly because each party considers it to be confidential and/or such direct sharing is prohibited by laws. Thus, secure *multi-party computation (MPC)* can be adopted. In MPC, the parties input confidential data and receive the computed results without accessing sensitive information from other parties. It has emerged as a significant area of interest in both research and practice as the importance of information security and privacy continues to grow.

To realize MPC, various privacy-preserving techniques such as *garbled circuits (GC)*, *secret sharing (SS)*, and *homomorphic encryption (HE)* are used to protect confidential data from being exposed, motivating more entities to participate in joint computation [16, 21, 66]. Although regulations restrict direct sharing of specific content, such as payment records, an efficient privacy-preserving graph analysis framework can enable collaborative computation without violating these laws. Previous works have relied on the data structure of a secret array of vertex/edge tuples, which is implemented with parallelized garbled circuits [43], or secret sharing with particular oblivious shuffle protocols [2], and the performances of both are primarily limited by the communication overhead.

In this article, we focus on secure graph data analysis involving two parties. Existing solutions [2, 43] choose to put all vertices and edges in garbling or sharing protocols, which successfully solves the important privacy challenge in secure graph analysis, but also brings a lower bound of $\Omega(|V|+|E|)$ communication to complete each iteration, where V and E denote vertices and edges, respectively. In contrast, our proposed method takes advantage of the fact that the graph data naturally arise from each party, so each party already knows

*Work is done during the research internship at Ant Group.



(a) Two banks/telecom operators (b) Two social-media websites

Figure 1: Examples of problem setting

its subgraph. With a novel data structure, ChaosTable, which addresses security challenges of using local graph data, our framework can utilize possible chances of performing parts of graph analysis computation locally on each party, and thus reduce the network traffic between the parties.

1.1 Problem Setting

The problem of two entities conducting joint graph analysis on their data is an abstraction of various real-world scenarios. For example, two banks wish to work together on detecting financial fraud. In this context, each bank account is visualized as a vertex, while each transaction forms an edge. Similarly, joint analysis can occur between two social-media websites, with each (properly anonymized) person depicted as a vertex and following/followed-by relationships as edges. Likewise, two telecom operators can cooperatively analyze a graph where vertices are phone numbers and edges are call records.

In such cases, each party inherently holds graph data originating from its regular business, which is an important fact utilized when designing our framework GraphAce. For example, a bank inherently knows about the accounts opened at it, the transaction records from/to these accounts, and the existence of other accounts associated with such records.

Let P_0 and P_1 represent the two parties. (When considering a party P_b, we use P_{1-b} to denote the other party.) Essentially, each party P_b holds a set of known vertices V_b and edges E_b to serve as input for their collaborative computation.

Vertex set. V_b denotes vertices that the party P_b knows. We clarify that $v \in V_b$ does not imply that P_b "owns" vertex v; rather, it merely indicates that P_b knows the existence of vertex v. For the financial example, V_0 includes all accounts opened at Bank 0, as well as any other accounts that have associated transaction records from/to any account opened at Bank 0 (so that Bank 0 knows them). For social media websites, V_b includes any person who registers on this website.

Edge set. We categorize the edges into cross-party ones and other normal edges based on whether each edge's establishment involves interaction between the two parties or not. For example, inter-bank transaction records are cross-party edges, while transactions within each bank are other normal edges; any following/followed-by relationship in a social-media website does not interact with the other website, and is considered as normal edges. Based on this, each party P_b manages a set of edges E_b for computation, including all normal edges it knows, and any cross-party edges that point to party P_b from P_{1-b}. This ensures that *any relationship is included exclusively in E_0 or E_1*, but not both.

Figure 1 illustrates concrete examples of V_b and E_b for the mentioned cases. In the financial context (Figure 1a), the digit in each vertex name signifies the bank at which the account was opened. An inter-bank transaction (e.g. from `0s` to `1k`) is known by both banks, but only one party should incorporate this edge. We stipulate that the bank receiving the transaction should include the edge. Similarly, for telecom operators, inter-company call records should only exist in the edge set submitted by the party receiving the call. In the context of social network websites (Figure 1b), all edges are normal ones and are submitted without elimination (note that if a person follows another person on both websites, two distinct "following" relationships are formed, e.g. from `b` to `k`).

In summary, we assume that each party P_b supplies a set of vertices V_b and a set of edges E_b, where V_b encompasses all vertices known by P_b, and E_b is constructed so that every relationship is counted exactly once in E_0 and E_1.

1.2 Our Contribution

Our contribution is highlighted in the following.

- We propose a new and efficient technical roadmap for secure two-party graph analysis, which utilizes local graph data from each party and avoids putting the whole graph into multi-party protocols, thus reducing the communication complexity. Following this roadmap, each analysis iteration only needs $\Theta(|V|)$ communication, in contrast to the previous lower bound $\Omega(|V|+|E|)$, while the computation becomes the same $\Theta(|V|+|E|)$ as insecure methods.
- To realize the roadmap, we identify security challenges in utilizing local graph data, namely *weight privacy* and *obliviousness*. Then we protect *weight privacy* with the mixed primitives system of secret sharing and homomorphic encryption, and achieve *obliviousness* with a novel data structure ChaosTable, which is carefully designed to fit in the above complexities without sacrificing security.
- Our designed framework, named GraphAce, improves usability by supporting *end-to-end* computation, in contrast to existing methods that only deal with already garbled or secret-shared graphs and require (not yet implemented or discussed) steps before and after iterations, while achieving security in the presence of semi-honest adversaries.

Finally, our evaluation shows that GraphAce achieves a speed of up to 14300× faster and a communication reduction of up to 55300× (depending on the network and the application), compared to the state-of-the-art secure two-party analysis framework GraphSC [43]. It also successfully processes a massive graph with more than 1 million vertices and 132

million edges in a reasonable time, which is the most extensive (128× larger than previously reported) secure two-party graph analysis task to the best of our knowledge.

The subsequent parts of this article are organized as follows. Section 2 provides fundamental background information on graph analysis and related work. Section 3 gives an overview of our framework. Section 4 introduces our designed protocols and algorithms. Section 5 provides discussions on comparing our framework with alternative methods. Section 6 presents the results of the experiments to evaluate our solution. Finally, Section 7 concludes this paper.

2 Background and Related Works

2.1 Graph Analysis

Graph analysis (or graph processing, graph analytics) refers to iterative computation tasks on a graph. During the process, edges are assumed to be immutable, whereas the weights on vertices may change across iterations. A graph analysis *framework* means that it can generally compute any type of analysis, rather than a specific application. Plaintext (insecure) graph analysis has been thoroughly researched over time. In addition to parallel in-memory approaches [44,55], there have been frameworks that utilize hard disks [51,71], spread over a cluster of servers [12,22,70], or a combination of both [50,59].

For secure multi-party scenarios, GraphSC [43] is a two-party graph analysis framework based on parallelized garbled circuits (GC), which implements graph analysis iterations with sorting and scanning passes over a garbled array of vertex/edge tuples. Araki et al. [2] further propose an approach based on a secret-shared tuple array. They introduce a shuffle protocol for three parties of honest majority, achieving a lower complexity than GraphSC. Section 5.1 provides a detailed comparison of our method and the existing solutions.

There are also graph analysis frameworks with significantly different security models, e.g., with trusted processors [18], or targeting differential privacy [25,39,40,63].

2.2 Mixed Primitives System

As the communication overhead is recognized as the speed bottleneck of multi-party computation, the mixed primitives system of secret sharing (SS) and homomorphic encryption (HE) can be adopted. Compared with using only SS, a mixed system can decrease communication, since HE operations can be performed locally in a party without network traffic. This is adopted in other multi-party tasks to reduce communication, such as decision trees [1, 14, 19, 38, 61], clustering [10, 41], and neural network inference [15, 24, 31, 42, 47, 49].

Secret sharing (SS). Additive SS can securely store a value between the parties. A value x is expressed as $[\![x]\!]_0$ and $[\![x]\!]_1$, where $x \equiv [\![x]\!]_0 + [\![x]\!]_1 \pmod{2^W}$, and each P_b holds $[\![x]\!]_b$. The index b is often omitted (becoming $[\![x]\!]$) if there is no ambiguity. Upon shared values, the parties can evaluate linear operations such as addition without communication, as well as multiplication based on beaver triples [5]. Arbitrary operations can be implemented by converting into boolean sharing, evaluating a binary circuit, and finally converting back [17].

Homomorphic encryption (HE). HE schemes [13,46] enable one to compute on encrypted values without gaining knowledge about the plaintext. For a private-key system, each party P_b generates a pair of public and secret keys $(\mathrm{pk}_b, \mathrm{sk}_b)$ and sends pk_b to P_{1-b}. A party can encrypt a value x with a public key $\mathsf{Encrypt}(\mathrm{pk}, x)$, and can decrypt a ciphertext c with the secret key $\mathsf{Decrypt}(\mathrm{sk}, c)$. Fully homomorphic encryption (FHE) provides multiple types of computational operation, such as addition $\boxplus$ and multiplication $\boxtimes$, while partially homomorphic encryption (PHE) supports only a single type (e.g. addition only). To protect sensitive data, encrypted values in GraphAce are typically stored separately with the secret key. The ciphertext of x is denoted as $[\![x]\!]^H_b$ to show that it is encrypted with pk_{1-b} and in possession of P_b. (Note that the superscript H means a HE ciphertext, while anything like $[\![x]\!]$ without H denotes a secret-shared value.)

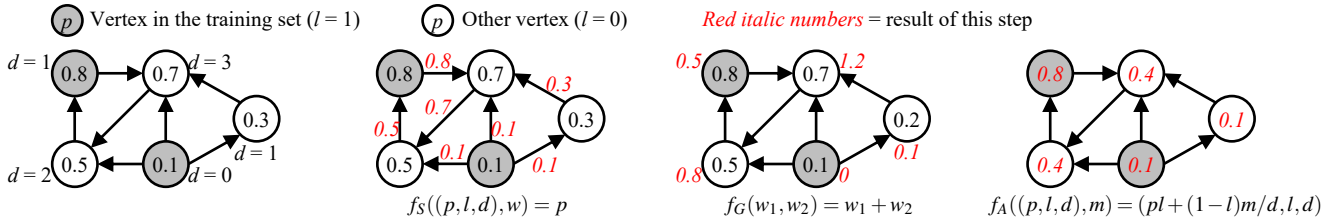
Conversion. The parties can convert secure data between HE and SS without leaking any sensitive information [24,38]. For notation, we use $\mathsf{A2H}_b$ to signify the protocol that inputs $[\![x]\!]$ and outputs $[\![x]\!]^H_b$ to party P_b only. $\mathsf{A2H}$ denotes the conversion from $[\![x]\!]$ to both parties ($[\![x]\!]^H_0$ and $[\![x]\!]^H_1$), and $\mathsf{H2A}_b$ denotes the conversion from $[\![x]\!]^H_b$ to $[\![x]\!]$. It is also possible to securely reveal an HE ciphertext $[\![x]\!]^H_b$ to P_b, so that P_b learns x and P_{1-b} learns nothing. We use $\mathsf{RevealHE}_b$ to denote this protocol. Appendix D gives formal descriptions of these protocols.

2.3 Other Related Works

Sangers et al. [53] introduce a multi-party PageRank method based on HE. However, it is specialized for PageRank and its evaluation is restricted to small graphs ($|V| \leq 1.6 \times 10^4$, $|E| \leq 2.56 \times 10^6$), while GraphAce is a generic framework and performs PageRank well on a significantly larger graph ($|V| = 1.5 \times 10^6$, $|E| = 1.32 \times 10^8$). Blanton et al. [6] suggest various data-oblivious graph algorithms like spanning trees and network flows, but its adjacency matrix structure makes it inefficient when analyzing large *sparse* graphs [8, 9, 32]. There are studies on privacy in other graph-related tasks, such as graph learning [3, 29, 33–35, 52, 54, 57, 60, 64, 65, 67, 69], knowledge graph reasoning [23, 62], minimal cut [68], graph database [11], and subgraph matching [26, 37, 56, 58].

3 Overview of GraphAce

Now we present an overview of GraphAce, beginning with formalizing our target task. We then introduce the rationale behind our motivation, the technical roadmap, and its realization. Finally, we conclude with a high-level workflow description.



(a) Initial state (b) Scatter (c) Gather (d) Apply

Figure 2: Example of performing semi-supervised label propagation (SSLP) with GAS abstraction

3.1 Graph Analysis Abstractions

As a graph analysis *framework*, GraphAce supports a wide variety of graph analysis applications by implementing the gather-apply-scatter (GAS) model [22] of graph computation, which was also adopted by existing secure graph analysis frameworks [2, 43]. The GAS model standardizes the *message passing* procedure between each vertex and its neighbors in each iteration into three steps: gather, apply, and scatter. So, a graph analysis application is characterized by three operators: gather f_G, apply f_A, and scatter f_S. Specifically,

- The **scatter** operator $f_S(w_v, w_e)$ determines the message to be sent along an edge, given the weight of the source vertex w_v and that of the current edge w_e.
- The **gather** operator $f_G(m_1, m_2)$ specifies how incoming messages at the same destination vertex are merged, i.e. messages m_1 and m_2 should be merged as $f_G(m_1, m_2)$. It is assumed that f_G must be associative and commutative.
- The **apply** operator $f_A(w_v, m)$ gives an updated weight of the vertex, based on the previous weight w_v and the gathered message m received by the current vertex.

Formally, given a graph of the vertex set V, the edge set E of tuples $(u, v, w(u,v))$ which denotes an edge from the vertex u to v of weight $w(u,v)$, and initial vertex weights $w^0(v)$ for $v \in V$, the vertex weights after t iterations will be

$$w^t(v) = f_A\left(w^{t-1}(v), \sum^{f_G}_{(u,v,w(u,v))\in E} f_S(w^{t-1}(u), w(u,v))\right) \tag{1}$$

where $\sum^{f_G}$ means reduction following the operator f_G. In practice, the total number of iterations is determined in an application-specific way. For example, in PageRank [45], this affects the precision and is typically 50 to 100 [30].

Figure 2 shows an example of semi-supervised label propagation (SSLP) [36]. In SSLP, the labels are probabilities of being in the positive class. Labels of vertices in the training set are known ($l = 1$), while those of the other vertices need to be predicted ($l = 0$). Each iteration can be implemented as: (Scatter) each vertex sends its probability p along each outgoing edge, (Gather) each vertex sums up received messages, and (Apply) each vertex, if $l = 0$, updates p by dividing the sum by its in-degree d. The operators are also exactly shown in Figure 2. Note that we pack the tuple (p, l, d) as the weight of each vertex, to ensure the purity of each operator.

3.2 Formalization

Porting the abstractions to our two-party setting (Section 1.1), we formalize secure two-party graph analysis as follows.

Input. Each party P_b initially holds a set of vertices V_b, initial weights $w^0_b(v)$ for each $v \in V_b$, and a (multi-)set of edges E_b. Every edge is represented by a triplet $(u, v, w(u,v))$, where $u, v \in V_b$, specifying an edge from vertex u to v with weight $w(u,v)$. Corresponding to the notation in plaintext computation (Section 3.1), the vertices and edges of the whole graph can be denoted as $V = V_0 \cup V_1$ and $E = E_0 \cup E_1$. V and E should remain unknown to any party, to preserve privacy.

Public information. The parties agree on the specification of the graph analysis application, f_I, f_G, f_A and f_S. The additional f_I is used to merge two weights if both parties have an initial value for a vertex (see "Output" below).

The number of iterations T is public. We also make public the values $\lceil \log_2 |V_0| \rceil$ and $\lceil \log_2 |V_1| \rceil$. This is in line with the standard practice in multi-party protocols, where it is permissible to reveal the data size to facilitate the computation (in addition, revealing $\lceil \log_2 |V_b| \rceil$ to P_{1-b} only gives a rough approximation of $|V_b|$ and nothing about E_b). The detailed difference between this and the public information of GraphSC (i.e., T and $|V| + |E|$) is further discussed in Appendix B.

Output. Each party P_b receives $w^T(v)$ for each $v \in V_b$, where

$$w^0(v) = \begin{cases} f_I(w^0_0(v), w^0_1(v)), & v \in V_0 \cap V_1, \\ w^0_b(v), & v \in V_b - V_{1-b}, \end{cases} \tag{2}$$

$$w^t(v) = f_A\left(w^{t-1}(v), \sum^{f_G}_{(u,v,w(u,v))\in E_0 \cup E_1} f_S(w^{t-1}(u), w(u,v))\right). \tag{3}$$

Security. Our framework is secure in the presence of semi-honest adversaries, ensuring that each party only acquires knowledge of public information, its input, its output, and any deductions derivable from the above information.

Assumptions to HE scheme. We assume that the adopted HE scheme satisfies *semantic security* (i.e. indistinguishability under chosen-plaintext attack, IND-CPA), and that the conversion between SS and HE can be securely implemented (i.e., computationally leaking no information; see Appendix D for details). Unlike many previous works that rely on a

certain HE scheme, we *do not require a specific scheme, even not limiting its type* (PHE or FHE), as long as it satisfies the above assumptions and the application-specific operators can be implemented on the ciphertexts. This gives chances of performance optimization, since we can choose a scheme as fast as possible based on the graph analysis application. For example, if no operator requires multiplication of ciphertexts, a PHE scheme can be adopted. In addition, future HE schemes that are more efficient and/or powerful can also be integrated, increasing the functional scalability of GraphAce.

3.3 Motivation

An important challenge of secure graph analysis is to prevent information leakage. Existing solutions address this by placing every vertex and edge in MPC protocols, incurring a lower bound of $\Omega(|V|+|E|)$ communication per iteration. However, insecure solutions can break this bound using local graph data. Thus, we adopt a different roadmap, starting from an insecure prototype and gradually improving its privacy.

Challenges of Secure Graph Analysis. In plaintext graph analysis, Equation 1 can be implemented directly "as-is", with a complexity of $\Theta(|V|+|E|)$ per iteration. On the other hand, a secure framework should (at least) protect *1) weight privacy*: each computing party should not obtain the plaintext of sensitive information (e.g., other party's input or intermediate results) and *2) obliviousness*: each party should not gain knowledge about sensitive information from the access patterns.[1] Although *weight privacy* can be naturally established with the adoption of common privacy-preserving techniques such as GC or SS [16, 21, 66], *obliviousness* imposes more challenges on graphs. Consider an application with $f_G(m_1,m_2)=m_1+m_2$, $f_A(w_v,m)=m$, $f_S(w_v,w_e)=w_v$ and $w^t(v)=\sum_{(u,v,w(u,v))\in E} w^{t-1}(u)$, which is referred to as "GraphAdd" in the following. If vertex y has three incoming edges from r, c and j, we should compute $w^t(\mathtt{y}) = w^{t-1}(\mathtt{r})+w^{t-1}(\mathtt{c})+w^{t-1}(\mathtt{j})$. *Direct* porting this onto general purpose secure circuit evaluation protocols, such as GC or SS, protects *weight privacy*, but violates *obliviousness* as it requires leaking information about E beforehand (i.e., there exist edges from r, c and j to y). This is because the access pattern of such a direct implementation is not data-oblivious (dependent on the secret input E). Consequently, secure frameworks must seek efficient data-oblivious algorithms, instead of direct implementation.

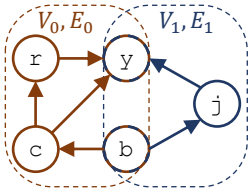
Previous Lower Bound of Communication. To address the above challenge, especially *obliviousness*, existing secure frameworks design alternative methods with access patterns independent of secret input E. GraphSC [43] adopts the data structure of a garbled array of $|V|+|E|$ tuples, where each tuple corresponds to a vertex or an edge. Each message passing iteration is organized as sorting and scanning passes with data-oblivious access patterns, so it can be realized by GC without information leakage. However, this also means that all vertices and edges must be sent to the MPC protocol (i.e. garbling). Since performing an iteration requires inspecting each vertex and edge with the MPC protocol at least once, working on the garbled array of tuples requires a lower bound of $\Omega(|V|+|E|)$ communication complexity per iteration. In fact, GraphSC needs $\Theta(\log^2(|V|+|E|))$ times over the bound due to the complexity of the sorting. Subsequent work by Araki et al. [2] is based on an SS array of tuples. The lower bound still holds, as it puts all vertices and edges in the MPC protocol (SS) as well. Although they develop efficient shuffle protocols instead of sorting, they can only bring it closer to, rather than breaking the lower bound of $\Omega(|V|+|E|)$ communication per iteration. This bound is not achieved in practice: even their honest majority setting needs an extra log-factor (Section 5.1). In summary, existing solutions achieve *obliviousness* by putting all vertices and edges in the MPC protocol and devising oblivious algorithms, but this makes all computation happen inside the MPC protocol and is limited by the lower bound of $\Omega(|V|+|E|)$ communication per iteration.

Inspiration: Utilizing Local Graph. As *not* noticed by previous secure solutions, we may get inspiration from distributed *plaintext* frameworks to design a two-party framework that can break the lower bound, since a distributed plaintext framework on two machines solves almost the same task as an *insecure* two-party framework. Unlike previous secure solutions [2, 43], distributed plaintext frameworks [12, 22, 50, 59, 70] do not require communication related to edges $|E|$ in iterations. To demonstrate the elimination of communication related to $|E|$, we now show an implementation of *insecure* two-party GraphAdd. We assume that the graph is spread over two parties and that each party P_b knows a set of vertices V_b and edges E_b (Figure 3a). Thus, P_0 can calculate and send $w^{t-1}(\mathtt{r})+w^{t-1}(\mathtt{c})$ to P_1, and P_1 sends $w^{t-1}(\mathtt{j})$ to P_0, so each can calculate $w^t(\mathtt{y})=(w^{t-1}(\mathtt{r})+w^{t-1}(\mathtt{c}))+w^{t-1}(\mathtt{j})$. The communication regarding y remains $O(1)$ even if y has more edges, as they only need to exchange a single number (the local sum) for y. As a result, the amount of communication is determined only by $|V|$, rather than by $|E|$. We notice a fundamental distinction: Insecure frameworks eliminate communication related to $|E|$ because each party utilizes its local graph, while current secure solutions [2, 43] did not take advantage of this. Thus, an essential *design goal* of GraphAce is to utilize the local graph. Nevertheless, it is still far from a complete framework satisfying privacy requirements, and we use Sections 3.4 to 3.6 to realize our roadmap to convert such a plaintext framework into a secure one.

3.4 Technical Roadmap of GraphAce

Unlike previous solutions that put all vertices and edges in the MPC protocol, our proposed GraphAce adopts a distinct

[1] Note that addressing these two privacy concerns does not mean that such a solution meets the security requirements (Section 3.2). On the other hand, any secure solution should at least protect against these two aspects.



(a) An example graph

P_0 receives

v	$p(v)$
b	8
c	5
r	7
y	2

P_1 receives

v	$p(v)$
b	8
j	3
y	2

(b) Revealed positions $p(v)$

P_0 puts	Position	P_1 puts
$[\![0_G]\!]_0^H$	1	$[\![0_G]\!]_1^H$
$[\![s_0^t(\mathtt{y})]\!]_0^H$	2	$[\![s_1^t(\mathtt{y})]\!]_1^H$
$[\![0_G]\!]_0^H$	3	$[\![s_1^t(\mathtt{j})]\!]_1^H$
$[\![0_G]\!]_0^H$	4	$[\![0_G]\!]_1^H$
$[\![s_0^t(\mathtt{c})]\!]_0^H$	5	$[\![0_G]\!]_1^H$
$[\![0_G]\!]_0^H$	6	$[\![0_G]\!]_1^H$
$[\![s_0^t(\mathtt{r})]\!]_0^H$	7	$[\![0_G]\!]_1^H$
$[\![s_0^t(\mathtt{b})]\!]_0^H$	8	$[\![s_1^t(\mathtt{b})]\!]_1^H$

(c) Using ChaosTable for CPI

Figure 3: An example of ChaosTable

technical roadmap. We first propose an insecure prototype framework (generalizing the above GraphAdd). Then we sequentially improve it to guarantee *weight privacy* and *obliviousness*, while being careful to avoid any communication related to $|E|$. Ultimately, we will obtain a secure two-party graph analysis framework which can break the $\Omega(|V|+|E|)$ lower bound and achieve $\Theta(|V|)$ communication per iteration.

The first insecure prototype is shown below.

Prototype 1 (Insecure two-party graph analysis). In this prototype framework, each iteration consists of two steps:

- **Local message passing (LMP).** Given $w^{t-1}(v)$ for $v \in V_b$, each party P_b computes $s_b^t(v)$ for $v \in V_b$, where $s_b^t(v)$ denotes the local sum of v in P_b, defined as

$$s_b^t(v) = \sum_{(u,v,w(u,v))\in E_b}^{f_G} f_S(w^{t-1}(u), w(u,v)). \quad (4)$$

  This step needs $\Theta(|E_b|)$ computation by the party P_b, but does not require any communication.
- **Cross-party interchange (CPI).** The parties exchange all computed values $s_b^t(v)$. Each party P_b then computes $w^t(v) = f_A\left(w^{t-1}(v), f_G(s_0^t(v), s_1^t(v))\right)$ for $v \in V_b$. This step requires $\Theta(|V|)$ computation and communication.

3.5 Protect Weights with Mixed Primitives

Prototype 1 violates *weight privacy* because the intermediate results $s_b^t(v)$ are computed and interchanged in plaintext.

To protect *weight privacy* in LMP, homomorphic encryption (HE) can be adopted while maintaining the same complexity and locality. Specifically, LMP is modified to input and output HE ciphertexts, i.e., given $[\![w^{t-1}(v)]\!]_b^H$ and resulting in $[\![s_b^t(v)]\!]_b^H$ for $v \in V_b$. (Note that collaborative protocols such as GC or SS are not suitable here because they introduce communication per edge, which violates our design goal.)

However, in CPI, we cannot directly send these HE ciphertexts. For example, if P_0 sent $[\![s_0^t(v)]\!]_0^H$ to P_1, P_1 would be able to decrypt it with the secret key and obtain the plaintext $s_0^t(v)$. For this context, the mixed primitives system (Section 2.2) is well suited, since CPI allows communication related to vertices $|V|$. CPI can be implemented by converting LMP results from HE ciphertexts to SS values, and the actual computation like f_A occurs in the form of SS. Finally, the SS results are converted back to HE, to facilitate LMP of the next iteration. The modified prototype is shown below.

Prototype 2 (Two-party graph analysis satisfying only weight privacy). Each iteration still consists of two steps:

- **Local message passing (LMP).** Given $[\![w^{t-1}(v)]\!]_b^H$ for $v \in V_b$, each party P_b computes $[\![s_b^t(v)]\!]_b^H$ for $v \in V_b$, as $[\![s_b^t(v)]\!]_b^H = \sum_{(u,v,w(u,v))\in E_b}^{f_G} f_S([\![w^{t-1}(u)]\!]_b^H, w(u,v))$.
- **Cross-party interchange (CPI).** The parties invoke H2A to convert each $[\![s_b^t(v)]\!]_b^H$ into $[\![s_b^t(v)]\!]_b$ (if P_b needs to provide $[\![s_b^t(v)]\!]_b^H$ but $v \notin V_b$, it provides $[\![0_G]\!]_b^H$ instead, where $f_G(0_G, x) = x$ for any x). Then they compute $[\![w^t(v)]\!] = f_A\left([\![w^{t-1}(v)]\!], f_G([\![s_0^t(v)]\!], [\![s_1^t(v)]\!])\right)$. Finally, they invoke A2H to convert each $[\![w^t(v)]\!]$ to $[\![w^t(v)]\!]_b^H$.

3.6 Guarantee Obliviousness with ChaosTable

Although the above protects *weight privacy*, so that each party cannot directly learn the input weights or intermediate results, the access patterns of the vertices during CPI still leak sensitive information. In Prototype 2, P_{1-b} asks P_b to provide a value for each vertex, thus leaking the information on whether P_{1-b} is interested in each vertex, which essentially leaks V_{1-b} to P_b.[2] Thus, we need to achieve *obliviousness*, especially to protect "which party knows which vertices", while the method should fit in the current $\Theta(|V|)$ complexity of CPI.

We carefully design the ChaosTable data structure, which effectively ensures *obliviousness* during CPI without violating the complexity requirement. It needs to be built before the first iteration and is used during the CPI of each iteration.

Building ChaosTable. The ChaosTable is built by assigning each vertex a random unique *position* within a linear range. Each party receives positions of known vertices, and each vertex known by both parties receives the same position for the two parties. Formally, the functionality is defined as follows.

Functionality 1 (Build ChaosTable).

- **Input.** Each party P_b inputs a set of vertices V_b.
- **Public information.** $\lceil \log_2 |V_0| \rceil$ and $\lceil \log_2 |V_1| \rceil$ are public.
- **Output.** Each party P_b receives a *position*, $p(v)$, for each vertex $v \in V_b$, satisfying Uniqueness: $p(u) = p(v) \Leftrightarrow u = v$, i.e., different vertices receive different positions, while *the two parties receives the same position $p(v)$ for $v \in V_0 \cap V_1$*, Privacy: Each party P_b should not learn anything beyond public information (especially sensitive information about V_{1-b}) from the received values $\{p(v)\}$, and Linearity: There

[2] If the prototype is modified reversely, making P_b provide the ciphertext that it knows to P_{1-b}, the leakage will persist as P_{1-b} can learn V_b.

Protocol or algorithm	1. Build ChaosTable	2. Build Adjacency Lists	3. Initial Weight Interchange (IWI)	4. Local Message Passing (LMP)	5. Cross-Party Interchange (CPI)	6. Reveal Results
Communication	$\Theta(\|V\|)$	0	$\Theta(\|V\|)$	0	$\Theta(\|V\|)$ per iteration	$\Theta(\|V\|)$
Computation	$\Theta(\|V\|\log\|V\|)$	$\Theta(\|E\|)$	$\Theta(\|V\|)$	$\Theta(\|E\|)$ per iteration	$\Theta(\|V\|)$ per iteration	$\Theta(\|V\|)$
Input	V_b	E_b	$\{w_b^0(v)\}, \{p(v)\}$	$\{[\![w^{t-1}(v)]\!]_b^H\}, \{L_b(v)\}$	$\{[\![s_b^t(v)]\!]_b^H\}, [\![W^{t-1}]\!], \{p(v)\}$	$\{[\![w^T(v)]\!]_b^H\}$
Output	$\{p(v)\}$	$\{L_b(v)\}$	$\{[\![w^0(v)]\!]_b^H\}, [\![W^0]\!]$	$\{[\![s_b^t(v)]\!]_b^H\}$	$\{[\![w^t(v)]\!]_b^H\}, [\![W^t]\!]$	$\{w^T(v)\}$

Repeat for $t = 1, 2, \ldots, T$

Figure 4: Workflow of GraphAce ($\{X(v)\}$ means $\{X(v) \text{ for } v \in V_b\}$)

exists a positive integer N computable from public information, satisfying $N = \Theta(|V|)$ and $p(v) \in [N]$ for any $v \in V$, where $V = V_0 \cup V_1$ and $[N]$ is the set $\{1, 2, \ldots, N\}$. This ensures a linear $\Theta(|V|)$ complexity of using ChaosTable.

Figure 3b shows examples of $p(v)$ revealed for the graph in Figure 3a with $N = 8$. Section 4.1 will introduce our method for realizing Functionality 1, while Section 5.2 will discuss the difference between our proposed method and possible alternatives. For now, let us assume that $\{p(v)\}$ are already computed and continue with the usage.

Using ChaosTable. The ChaosTable contains N *slots* for each party, so one can put data in the ChaosTable according to the positions $p(v)$, to interchange results. Specifically, the data related to the vertex v should appear in the $p(v)$-th slot. For each iteration, the modified cross-party interchange (CPI) makes use of ChaosTable in the five steps below. Note that we aim to provide a descriptive demonstration here, while the implementation details will be introduced in the corresponding protocols that use ChaosTable (Sections 4.3 and 4.5).

1. **Put data in.** Each party P_b locally puts its data to interchange with respect to v (i.e. $[\![s_b^t(v)]\!]_b^H$) in the $p(v)$-th slot, for each $v \in V_b$, as shown in Figure 3c. For a slot that P_b has no vertex to occupy, an empty value $[\![0_G]\!]_b^H$ is provided.
2. **Convert to SS.** The parties convert all the above HE ciphertexts into SS values with H2A. The security of H2A ensures that each party P_b cannot distinguish whether P_{1-b} provided a real encrypted message $[\![s_{1-b}^t(v)]\!]_{1-b}^H$ or a default $[\![0_G]\!]_{1-b}^H$, thus protecting secret information V_{1-b}.
3. **Compute collaboratively.** The parties jointly evaluate the main computational part of CPI in the form of SS, i.e., $[\![w^t(v)]\!] = f_A\left([\![w^{t-1}(v)]\!], f_G([\![s_0^t(v)]\!], [\![s_1^t(v)]\!])\right)$. Note that this step (as well as the above and below conversions) is performed exhaustively for *all* N slots, thus hiding the information of "which party is interested in which vertices (slots)". Although this seems like a waste of computation, *Linearity* ensures that the complexity remains $\Theta(|V|)$.
4. **Convert to HE.** For each slot, the parties convert the resulting SS values above into HE ciphertexts with A2H.
5. **Pick out results.** Each party P_b locally picks the result of this iteration t (i.e. $[\![w^t(v)]\!]_b^H$) from the $p(v)$-th slot of the above conversion results. After this step, each party gathers enough data for the LMP of the next iteration.

As a result, the above steps successfully address concerns about *obliviousness*. The final prototype is Prototype 3.

Prototype 3 (GraphAce). Each iteration consists of **local message passing** (**LMP**, same as Prototype 2) and **cross-party interchange** (**CPI**, implemented by the above five steps).

3.7 End-to-End Workflow of GraphAce

Now we introduce the workflow of GraphAce, expanding Prototype 3, as shown in Figure 4. The framework starts from building the ChaosTable (Section 4.1), resulting in the positions $p(v)$. This part involves $\Theta(|V|)$ communication with recent secure merge protocols. Each party P_b locally builds adjacency lists $L_b(v)$ for $v \in V_b$ in $\Theta(|E_b|)$ time without communication (Section 4.2), to be used in local message passing. Before iterations, we also execute a protocol named Initial Weight Interchange (IWI, Section 4.3) with $\Theta(|V|)$ communication, using the ChaosTable, so that each party P_b obtains the initial weights as HE ciphertexts, i.e. $[\![w^0(v)]\!]_b^H$ for $v \in V_b$. The parties also hold an arithmetic-shared version of the initial weights, $[\![W^0]\!]$, indexed by the positions of ChaosTable (i.e., $p(v)$), for future use. Formally, we define

$$W^t(x) = \begin{cases} w^t(v), & (\exists v)(p(v) = x), \\ \text{undefined (any value is fine)}, & \text{otherwise.} \end{cases} \tag{5}$$

The iterations hold the invariance that the t-th iteration starts from the values $[\![w^{t-1}(v)]\!]_b^H$ for $v \in V_b$ and $[\![W^{t-1}]\!]$, producing $[\![w^t(v)]\!]_b^H$ for $v \in V_b$ and $[\![W^t]\!]$. Following the specifications of Prototype 3, each iteration consists of local message passing (LMP, Section 4.4) and cross-party interchange (CPI, Section 4.5). In LMP, each party is given $[\![w^{t-1}(v)]\!]_b^H$ and computes $[\![s_b^t(v)]\!]_b^H$ for $v \in V_b$. This can be implemented with $\Theta(|E_b|)$ computation without communication. Based on $[\![s_b^t(v)]\!]_b^H$ and ChaosTable, we then perform CPI to obtain $[\![w^t(v)]\!]_b^H$ for $v \in V_b$, as well as $[\![W^t]\!]$, to prepare for the next iteration. This only costs $\Theta(|V|)$ communication per iteration.

Finally, the HE results $[\![w^T(v)]\!]_b^H$ for $v \in V_b$ are revealed to plaintext $w^T(v)$ (Section 4.6), with $\Theta(|V|)$ communication.

4 Protocols and Algorithms

This section presents the protocols to finalize the workflow. The correctness and complexity are examined in the descriptions, while the security is analyzed in Appendix A.

Algorithm 1 BuildChaosTable

Requires: Each P_b holds V_b.
Ensures: Each P_b obtains $p(v)$ for $v \in V_b^+$ (see below).

$N \leftarrow 2^{1+\max(\lceil \log_2 |V_0| \rceil, \lceil \log_2 |V_1| \rceil)}$
Each P_b prepares an array A_b of N pairs (vertex, toReveal):
  $V_b^+ \leftarrow V_b \cup \{(N/2 - |V_b|) \text{ different dummy vertices}\}$
  for all $v \in V_b^+$ **do**
    Append $(v,0)$ and $(v,1)$ to A_b
  Sort A_b by first "vertex" then "toReveal"
Each P_b shares A_b, resulting $[\![A_0]\!]$ and $[\![A_1]\!]$
$[\![A]\!], [\![\text{trace}]\!] \leftarrow \mathsf{SecureMerge}([\![A_0]\!], [\![A_1]\!])$
  // Also by first "vertex" then "toReveal"
$[\![R]\!] \leftarrow \mathsf{GenerateRandomPermutation}([\![[1,2,\ldots,N]]\!])$
$[\![L]\!] \leftarrow$ array of $2N$ elements, each element $[\![L[i]]\!] = [\![R[\lceil i/2 \rceil]]\!]$
$[\![L_0]\!], [\![L_1]\!] \leftarrow \mathsf{RevertSecureMerge}([\![L]\!], [\![\text{trace}]\!])$
for all $b \in \{0,1\}$ **do**
  for all $i \in [N/2]$ **do**
    Invoke $\mathsf{A2H}_b$ with $[\![L_b[2i]]\!]$, so P_b receives $[\![L_b[2i]]\!]_b^H$
    Invoke $\mathsf{RevealHE}_b$ with $[\![L_b[2i]]\!]_b^H$, so P_b receives $L_b[2i]$
Each P_b locally computes:
  for all $i \in [N/2]$ **do**
    $p(A_b[2i].\text{vertex}) \leftarrow L_b[2i]$

4.1 Building ChaosTable

Algorithm 1 shows our protocol for building the ChaosTable, realizing Functionality 1. Each party P_b starts by computing $N = 2^{1+\max(\lceil \log_2 |V_0| \rceil, \lceil \log_2 |V_1| \rceil)}$ and deterministically padding its input to be $N/2$ vertices, so that P_{1-b} cannot learn the exact number of $|V_b|$. Each P_b then builds and sorts an array A_b of N pairs, where each vertex corresponds to two pairs. Figure 5 shows a concrete example given the input from Figure 3a, assuming that P_1 adds a dummy vertex z. It is important that *every* pair will be *assigned* a secret-shared position, but only the positions of pairs with toReveal $= 1$ will be *revealed*.

Then, the collective part starts to assign the positions, where each party P_b shares the array as $[\![A_b]\!]$, and they merge two sorted arrays together (e.g., by odd-even merge [4] or linear complexity merge [7]). Consider the plaintext that corresponds to the merged array A. A vertex that is exclusively in either V_0 or V_1 (e.g. c, j, r and z) matches two consecutive pairs in A: the first pair has a toReveal bit 0, and the second pair has a toReveal bit 1. For a vertex v in $V_0 \cap V_1$ (e.g. b and y), it is represented by four consecutive pairs in A: the first two pairs have toReveal bits 0, while the subsequent two pairs have toReveal bits 1.

To make the positions random, we generate a random permutation R of $[N]$, repeating each element so that it becomes an array L of $2N$ elements, which are the positions assigned to each element of A. For a vertex exclusively known to one party, the two pairs receive the same position. For a vertex known by both parties, the two pairs with toReveal $= 0$ receive the same position and the two with toReveal $= 1$ receive another same position. We then revert the merge but apply it to L (i.e., there exists a permutation π such that $\pi(A_0||A_1) = A$ and $\pi^{-1}(L) = L_0||L_1$), so that L_0 and L_1 are the positions assigned to each element of A_0 and A_1, respectively.

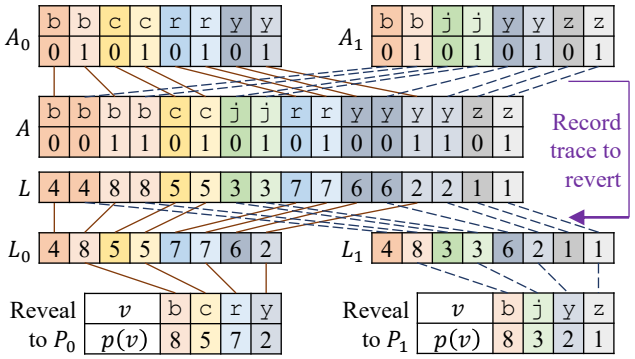


Figure 5: An example illustration of BuildChaosTable

Finally, we reveal to P_b the *even-indexed* elements of L_b, corresponding to the pairs in A_b with toReveal $= 1$, and each party can connect the received positions to the vertices.[3] Other SS values, such as A, L, and *odd-indexed* elements of each L_b, are not revealed to any party for privacy. In the following, we show that the results satisfy the requirements of Section 3.6.

Theorem 1. $\{p(v)\}$ *satisfy Uniqueness (Functionality 1).*

Proof. We divide A into N groups, each of two pairs. As A is sorted and each vertex appears twice or four times, the field "vertex" in each group must be equal and L assigns each group a unique position. For $u \neq v$, they are in different groups of A, thus receiving different positions. For $u = v$, we only need to show that both parties receive the same $p(v)$ if $v \in V_0 \cap V_1$. Consider the four pairs of v in A, forming a group of two $(v,0)$ pairs that receive the same position and two $(v,1)$ that receive another position. Since the positions corresponding to $(v,1)$ in L_0 and L_1 come from the same group, the values revealed to both parties are the same. □

Theorem 2 (Privacy)**.** *For any input (V_0, V_1), the $N/2$ values that* BuildChaosTable *reveals to each party P_b follow the distribution of uniformly randomly choosing $N/2$ numbers from $[N]$ (i.e., ordered samples without replacement). (Thus, the revealed values contain no information on private input.)*

Proof. Without loss of generality, consider the $N/2$ values revealed to P_0. The values are derived from L_0, originate from L, and are ultimately from $N/2$ indices of R. These indices are distinct due to Theorem 1 (otherwise, two different vertices receive the same position) and are determined solely by the inputs (V_0, V_1) since the merging is independent of the randomness of R. Therefore, for any (V_0, V_1), the $N/2$ values revealed to P_0 are analogous to selecting $N/2$ elements from R

[3] We use A2H then RevealHE here to simplify the security analysis; directly revealing the shares also works, as it does not leak more information.

Algorithm 2 BuildAdjLists

Requires: Each P_b holds E_b.
Ensures: Each P_b obtains $L_b(v)$ for $v \in V_b$.

Each P_b executes:
  for all $v \in V_b$ **do**
    $L_b(v) \leftarrow$ empty list of pairs
  for all $(u,v,w(u,v)) \in E_b$ **do**
    Append $(u,w(u,v))$ to $L_b(v)$

based on the determined indices. As R is a uniformly random permutation of $[N]$, this selection process mirrors the choice of ordered samples $N/2$ without replacement of $[N]$. □

Theorem 3 (Linearity)**.** *For any input* (V_0,V_1)*, (1)* N *is computable from public information, and (2)* $N = \Theta(|V|)$.

Proof. (1) is trivial, since $\lceil \log_2 |V_0| \rceil$ and $\lceil \log_2 |V_1| \rceil$ are public and $N = 2^{1+\max(\lceil \log_2 |V_0| \rceil, \lceil \log_2 |V_1| \rceil)}$. (2) holds as $|V| \leq 2 \cdot \max(|V_0|,|V_1|) \leq N \leq 2 \cdot \max(2|V_0|,2|V_1|) \leq 4 \cdot |V|$. □

For complexity, each party needs $O(N \log N)$ local computation to construct A_b, while the communication depends on the merge protocol, e.g. $\Theta(N)$ [7] or $\Theta(N \log N)$ [4]. Finally, each party needs $\Theta(N)$ local computation.

We can replace N with $|V|$ in complexities, since we proved $N = \Theta(|V|)$, so that it aligns with the notation of existing frameworks (note that the exact value $|V|$ *remains unknown* to any party). Thus, BuildChaosTable needs $O(|V| \log |V|)$ computation and (possibly) $\Theta(|V|)$ communication.

4.2 Building Local Adjacency Lists

For efficient computation in iterations, each P_b processes an adjacency list $L_b(v) = \{(u,w(u,v)) | (u,v,w(u,v)) \in E_b\}$ for each $v \in V_b$, as shown in Algorithm 2, which are used in local message passing (Section 4.4). The correctness of this algorithm is trivial. This step requires $\Theta(|E_b|)$ computation by party P_b and no communication.

4.3 Initial Weight Interchange (IWI)

Although the motivation of ChaosTable is to achieve *obliviousness* during cross-party interchange (CPI), it is also suitable to securely compute the initial vertex weights defined in Equation 2, given the input weights $w_b^0(v)$ from the parties, with the difference that the parties initially hold plaintext input $w_b^0(v)$ instead of the HE ciphertexts. To protect privacy, the result is only represented in secret forms, namely an HE version $[\![w^0(v)]\!]_b^H$ and an SS version $[\![W^0]\!]$ (Equation 5).

Algorithm 3 shows the protocol IWI (Initial Weight Interchange). Each party P_b starts by placing its input $w_b^0(v)$ in the ChaosTable according to $p(v)$. Each untouched slot possibly corresponds to a dummy vertex, a vertex exclusively known by the other party, or no vertex at all (and P_b cannot

Algorithm 3 IWI (Initial Weight Interchange)

Requires: Each P_b holds $w_b^0(v)$ and $p(v)$ for $v \in V_b$.
Ensures: Each P_b obtains $[\![w^0(v)]\!]_b^H$ for $v \in V_b$. Parties hold $[\![W^0]\!]$.

Each P_b prepares an array W_b^0 of length N:
  for all $v \in V_b$ **do**
    $W_b^0[p(v)] \leftarrow w_b^0(v)$
  Fill the other positions of W_b^0 with 0_I
Each P_b shares W_b^0, resulting $[\![W_0^0]\!]$ and $[\![W_1^0]\!]$
$[\![W^0]\!] \leftarrow f_I([\![W_0^0]\!],[\![W_1^0]\!])$
Invoke A2H with $[\![W^0]\!]$, so each P_b receives $[\![W^0]\!]_b^H$
Each P_b locally computes the result as:
  for all $v \in V_b$ **do**
    $[\![w^0(v)]\!]_b^H \leftarrow [\![W^0[p(v)]]\!]_b^H$

distinguish whether P_{1-b} has any vertex for this position, as in Section 4.1). Thus, we require P_b to provide an empty value for each position to which it does not have a vertex to correspond, so it would not affect the potential results. Since we need to evaluate f_I for each vertex, the empty value is 0_I here, where 0_I satisfies that $f_I(0_I,x) = f_I(x,0_I) = x$, for any x.[4]

After that, the parties convert the input data into SS values and evaluate f_I on the shares (for each position), resulting in the needed $[\![W^0]\!]$. The SS values are then converted to HE with A2H, and each party can get the initial weights (as HE ciphertext) for each vertex according to the positions $p(v)$.

Correctness proof. Let us first prove the correctness of $[\![W^0]\!]$.

For $v \in V_0 \cap V_1$, each P_b inputs $W_b^0[p(v)] = w_b^0(v)$. The corresponding plaintext that it computes becomes $W^0[p(v)] \leftarrow f_I(w_0^0(v), w_1^0(v))$, which correctly computes $W^0[p(v)]$ according to the definitions of $w^0(v)$ and W^0 (Equations 2 and 5).

For $v \in V_b - V_{1-b}$, P_b inputs $W_b^0[p(v)] = w_b^0(v)$ while P_{1-b} inputs the corresponding $W_b^0[p(v)] = 0_I$ (note that P_{1-b} does not know $p(v)$; it only inputs 0_I for each empty slot). The corresponding plaintext becomes $W^0[p(v)] \leftarrow f_I(w_0^0(v), 0_I)$ if $b = 0$ or $W^0[p(v)] \leftarrow f_I(0_I, w_1^0(v))$ if $b = 1$, resulting in $w_b^0(v)$. This correctly computes $W^0[p(v)]$ according to definitions.

For any other position x that does not have a vertex, Equation 5 allows us to compute any value (actually 0_I here).

Given the correctness of $[\![W^0]\!]$ and its definition (Equation 5), the correctness of $[\![w^0(v)]\!]_b^H$ is directly established. □

Now we analyze the complexity of IWI. Each party first needs $\Theta(N)$ local computation. The collective part involves the communication and computation of $\Theta(N)$. Finally, each party needs a local computation of $\Theta(N)$. In summary, IWI needs $\Theta(|V|)$ computation and communication. Note that this algorithm only needs $O(1)$ depth, so it is fully parallelizable.

[4] This existence of 0_I simplifies the design of IWI. If such 0_I does not "naturally" exist, we can let any value that is impossible to appear as the input $w_b^0(v)$, be 0_I, and modify f_I into an if-then-else style to be aware of this 0_I.

Algorithm 4 LMP (Local Message Passing)

Requires: Each P_b holds $[\![w^{t-1}(v)]\!]_b^H$ and $L_b(v)$ for $v \in V_b$.
Ensures: Each P_b obtains $[\![s_b^t(v)]\!]_b^H$ for $v \in V_b$.

Each P_b executes:
  for all $v \in V_b$ **do**
    $[\![s_b^t(v)]\!]_b^H \leftarrow \sum_{(u,w)\in L_b(v)}^{f_G} f_S([\![w^{t-1}(u)]\!]_b^H, w)$

Algorithm 5 CPI (Cross-Party Interchange)

Requires: Each P_b holds $[\![s_b^t(v)]\!]_b^H$ and $p(v)$ for $v \in V_b$.
Parties hold $[\![W^{t-1}]\!]$.
Ensures: Each P_b obtains $[\![w^t(v)]\!]_b^H$ for $v \in V_b$. Parties hold $[\![W^t]\!]$.

Each P_b prepares an array $[\![S_b^t]\!]_b^H$ of length N:
  for all $v \in V_b$ **do**
    $[\![S_b^t[p(v)]]\!]_b^H \leftarrow [\![s_b^t(v)]\!]_b^H$
  Fill the other positions of $[\![S_b^t]\!]_b^H$ with $[\![0_G]\!]_b^H$
Invoke $\mathsf{H2A}_b$ with each $[\![S_b^t]\!]_b^H$, resulting $[\![S_0^t]\!]$ and $[\![S_1^t]\!]$
$[\![W^t]\!] \leftarrow f_A([\![W^{t-1}]\!], f_G([\![S_0^t]\!], [\![S_1^t]\!]))$
Invoke A2H with $[\![W^t]\!]$, so each P_b receives $[\![W^t]\!]_b^H$
Each P_b locally computes the result as:
  for all $v \in V_b$ **do**
    $[\![w^t(v)]\!]_b^H \leftarrow [\![W^t[p(v)]]\!]_b^H$

4.4 Local Message Passing (LMP)

Given $\{[\![w^{t-1}(v)]\!]_b^H\}$, namely the vertex weights after the previous iteration (which is computed in IWI, Section 4.3, or CPI, Section 4.5), Algorithm 4 shows the Local Message Passing (LMP) procedure, which transforms its input into local partial results $[\![s_b^t(v)]\!]_b^H$ (Equation 4), for $v \in V_b$. This can be easily implemented, given the local adjacency lists $L_b(v)$, for $v \in V_b$.

The correctness is given by the definition of $s_b^t(v)$ (Equation 4). LMP requires $\Theta(|E_b|)$ computation by P_b and no communication. Note that the depth is $O(\log |E_b|)$ (actually logarithm to the maximum degree), so it can be well parallelized.

4.5 Cross-Party Interchange (CPI)

After obtaining the partial results computed locally in LMP, namely $\{[\![s_b^t(v)]\!]_b^H\}$, we merge them for each party, preparing $\{[\![w^t(v)]\!]_b^H\}$ and $[\![W^t]\!]$ to be used for the next iteration. This process is similar to IWI, which also computes a function on vertices using ChaosTable. However, the input here is in the form of HE ciphertext instead of plaintext.

Algorithm 5 shows the protocol of CPI (Cross-Party Interchange). Each party still begins by putting the data in the ChaosTable, but each element is an HE ciphertext, and the empty value is an encrypted $[\![0_G]\!]_b^H$, so it has no effects with regard to f_G. The parties then invoke H2A, converting the ciphertexts into SS values for the next collective computations.

The collective part includes evaluating f_G followed by f_A. It is important to note that f_G is evaluated just once per vertex here, regardless of the number of edges connected to it. This is due to the fact that many of the operations gathering f_G have already occurred locally in LMP (in the forms of HE), as described previously in Section 3.3. In general, because f_A might require the weight of each vertex from the previous iteration, denoted as $[\![W^{t-1}]\!]$, this weight is stored and passed as a parameter from the previous IWI or CPI protocol.

The final part is the same as IWI. The parties call A2H and pick the results from ChaosTable according to the positions.

Algorithm 6 RevealResults

Requires: Each P_b holds $[\![w^T(v)]\!]_b^H$ for $v \in V_b$.
Ensures: Each P_b obtains $w^T(v)$ for $v \in V_b$.

Each P_b executes:
  for all $v \in V_b$ **do**
    Participate in $\mathsf{RevealHE}_b$ with $[\![w^T(v)]\!]_b^H$, receiving $w^T(v)$
  for all $i \in [N/2 - |V_b|]$ **do**
    Participate in $\mathsf{RevealHE}_b$ with $[\![0_I]\!]_b^H$
  for all $i \in [N/2]$ **do**
    Participate in $\mathsf{RevealHE}_{1-b}$

Correctness proof. Similarly, we first prove the correctness of $[\![W^t]\!]$. For $v \in V_0 \cap V_1$, each P_b inputs $[\![S_b^t[p(v)]]\!]_b^H = [\![s_b^t(v)]\!]_b^H$. The corresponding plaintext becomes

$$\begin{aligned} &W^t[p(v)] \leftarrow f_A(W^{t-1}[p(v)], f_G(s_0^t(v), s_1^t(v))) \\ &= f_A(W^{t-1}[p(v)], f_G(\textstyle\sum_{(u,v,w(u,v))\in E_0}^{f_G} f_S(w^{t-1}(u), w(u,v)), \\ &\qquad\qquad \textstyle\sum_{(u,v,w(u,v))\in E_1}^{f_G} f_S(w^{t-1}(u), w(u,v)))) \\ &= f_A(W^{t-1}[p(v)], \textstyle\sum_{(u,v,w(u,v))\in E_0\cup E_1}^{f_G} f_S(w^{t-1}(u), w(u,v))) \end{aligned}$$

which correctly computes $W^t[p(v)]$ according to the definitions of $w^t(v)$ and W^t (Equations 3 and 5).

For $v \in V_b - V_{1-b}$, P_b inputs $[\![S_b^t[p(v)]]\!]_b^H = [\![s_b^t(v)]\!]_b^H$ while P_{1-b} inputs the corresponding $[\![S_{1-b}^t[p(v)]]\!]_{1-b}^H = [\![0_G]\!]_{1-b}^H$ (still, P_{1-b} does not know $p(v)$ but only puts $[\![0_G]\!]_{1-b}^H$ for each empty slot). The corresponding plaintext is $W^t[p(v)] \leftarrow f_A(W^{t-1}[p(v)], f_G(s_b^t(v), 0_G))$. Thus, $f_G(s_b^t(v), 0_G) = s_b^t(v)$ and we have $s_b^t(v) = \sum_{(u,v,w(u,v))\in E_b}^{f_G} f_S(w^{t-1}(u), w(u,v)) = \sum_{(u,v,w(u,v))\in E_0\cup E_1}^{f_G} f_S(w^{t-1}(u), w(u,v))$, as E_{1-b} does not contain any edge about v (as $v \notin V_{1-b}$). Thus, it correctly computes $W^t[p(v)]$ according to the definitions.

For any other position x, we are allowed to obtain any value.

For $[\![w^t(v)]\!]_b^H$, the correctness is directly given by the correctness of $[\![W^t]\!]$ and the definition of W^t. □

The complexity of CPI is similar to IWI. CPI needs $\Theta(|V|)$ computation and communication of $O(1)$ depth.

4.6 Reveal the Results

After the CPI for the last iteration, each party already obtains the final weight for each vertex, but only as HE ciphertext.

Table 1: Comparison of secure graph analysis frameworks

Framework	Communication	Computation	End to end	Data Structure	Setting
GraphSC [43]	$\Theta((\lvert V\rvert+\lvert E\rvert)\log^2(\lvert V\rvert+\lvert E\rvert))$		No	Garbled array of vertex/edge tuples	2-party (dishonest majority)
Araki et al. [2]	First: $\Theta((\lvert V\rvert+\lvert E\rvert)\log(\lvert V\rvert+\lvert E\rvert))$ Other: $\Theta((\lvert V\rvert+\lvert E\rvert)\log B)$		No	Secret-shared array of vertex/edge tuples	3-party, honest majority
Ours, GraphAce	$\Theta(\lvert V\rvert)$ *	$\Theta(\lvert V\rvert+\lvert E\rvert)$	Yes	ChaosTable and local adjacency lists	2-party (dishonest majority)

All complexities are reported per iteration. B = upper bound of the vertex degrees, which becomes $|E|$ in the worst case, to prevent leakage.
* The complexity for *the first* iteration depends on the secure merge protocol, which can achieve $\Theta(|V|)$ if a linear complexity merge [7] is adopted.

Thus, we need a final procedure to reveal the encrypted values to each party, which mainly calls RevealHE to do this securely. Algorithm 6 describes the RevealResults protocol. Each party P_b participates in $\mathsf{RevealHE}_b$ and $\mathsf{RevealHE}_{1-b}$, each for $N/2$ times. This reveals more numbers than the $|V_b|$ values that are actually needed by P_b to hide the exact number of vertices.

The $N/2$ invocations of $\mathsf{RevealHE}_{1-b}$ solely help P_{1-b} learn $N/2$ numbers, while P_b itself learns nothing. The $N/2$ invocations of $\mathsf{RevealHE}_b$ mix $|V_b|$ desired ciphertext to reveal and $N/2-|V_b|$ dummy values of $[\![0_I]\!]_b^H$. Although the protocol can be modified to allow P_b to randomly shuffle the actual and dummy values, to enhance randomness, the security of the current version already suffices (Appendix A).

The correctness is trivial given the correctness of RevealHE. This part involves communication and computation of $\Theta(|V|)$.

4.7 Putting Together

Algorithm 7 shows the protocol of GraphAce, implementing the entire workflow (Figure 4). We build the ChaosTable and local adjacency lists and then call IWI once. For iterations, we perform LMP and CPI alternatively, each for T times. Finally, we call RevealResults so each party receives its final output. The correctness is derived from that of each protocol.

Algorithm 7 Overall Protocol of GraphAce

Requires: Each P_b holds V_b, E_b and $w_b^0(v)$ for $v \in V_b$.
Ensures: Each P_b obtains $w^T(v)$ for $v \in V_b$.

Each P_b executes:
  $\{p(v)\} \leftarrow \mathsf{BuildChaosTable}(V_b)$
  $\{L_b(v)\} \leftarrow \mathsf{BuildAdjLists}(E_b)$
  $\{[\![w^0(v)]\!]_b^H\}, [\![W^0]\!] \leftarrow \mathsf{IWI}(\{w_b^0(v)\}, \{p(v)\})$
  for all $t \in [T]$ **do**
    $\{[\![s_b^t(v)]\!]_b^H\} \leftarrow \mathsf{LMP}([\![w^{t-1}(v)]\!]_b^H, \{L_b(v)\})$
    $\{[\![w^t(v)]\!]_b^H\}, [\![W^t]\!] \leftarrow \mathsf{CPI}(\{[\![s_b^t(v)]\!]_b^H\}, \{p(v)\}, [\![W^{t-1}]\!])$
  $\{w^T(v)\} \leftarrow \mathsf{RevealResults}(\{[\![w^T(v)]\!]_b^H\})$

5 Discussion

Now we compare GraphAce with prior works, and discuss alternative methods to build ChaosTable.

5.1 Comparison with Existing Solutions

We compare GraphAce with existing frameworks based on complexity and usability, as shown in Table 1. We assume that each call to any operator (f_G, f_A, f_S) has $O(1)$ complexity.

Communication. GraphSC [43] uses an array of $|V|+|E|$ garbled tuples where each element corresponds to a vertex or an edge, and each iteration is implemented with sorting and scan passes. Although it devises a parallel scan, sorting remains the bottleneck, resulting in $\Theta((|V|+|E|)\log^2(|V|+|E|))$ communication per iteration. Araki et al. [2] only requires sorting in the first iteration, which is implemented as reveal-sort of $\Theta((|V|+|E|)\log(|V|+|E|))$ complexity. For subsequent iterations, they introduce an efficient shuffle protocol, but this only works in a three-party setting with an honest majority and is not suitable for a two-party scenario. The unoptimized scan part still needs $\Theta((|V|+|E|)\log B)$ communication and becomes the bottleneck;[5] B is the upper limit of vertex degree, which becomes $|E|$ in the worst case to prevent leakage. Unlike previous solutions, we completely eliminate communication related to edges. GraphAce only needs $\Theta(|V|)$ communication for each iteration, while preserving the same setting with GraphSC (2-party, naturally with dishonest majority).

Computation. Previous solutions have the same complexity of computation and communication, since they put all graph data into the MPC protocol (GC or SS). In contrast, the computation complexity of GraphAce is greater than its communication complexity. However, this still surpasses existing secure frameworks and achieves *the same* computational complexity $\Theta(|V|+|E|)$ as plaintext (insecure) counterparts.

End-to-end usability. Existing studies assume that the input is already organized as a garbled or secret-shared array of tuples. Although this suits for *outsourced* computing, it does not *directly* solve the cooperative graph analysis problem, where each party holds a subgraph of vertices and edges (Section 1.1). Extra steps are required to transform the input into the array-of-tuple structure, which are nontrivial due to the structure's requirements, e.g., each vertex should appear *exactly once* in the array. Such pre-processing is not implemented or thoroughly discussed in existing articles. This also applies to post-processing, i.e. to obtain the results for each party from the final structure. In contrast, GraphAce starts with the input

[5] Although it can perform a sequential scan of $\Theta(|V|+|E|)$ communication instead of a parallel scan, its performance decreases largely as the depth increases to $\Theta(|V|+|E|)$. Thus, they practically need a factor of $\Theta(\log B)$.

of the vertices and edges of each party and finally outputs the resulting weights, which solves the end-to-end task of secure two-party graph analysis and improves usability.

5.2 Alternative Methods to Build ChaosTable

We now compare our proposed method of ChaosTable (Section 4.1) with possible alternative methods.

Private set intersection or union (PSI/PSU) cannot be directly adopted to implement Functionality 1, because such protocols reveal the contents of the intersection/union (i.e. $V_0 \cap V_1$ or $V_0 \cup V_1$), violating the requirement of *Privacy*, even adopting implementations that only reveal the size of the results (i.e. $|V_0 \cap V_1|$ or $|V_0 \cup V_1|$), e.g., each party gains knowledge about the proportion of common vertices.

Oblivious maps are capable to assign the vertex positions $p(v)$, which can be implemented as a (distributed) oblivious RAM (ORAM). However, the large domain of all possible vertices prohibits such direct usage. For example, if each vertex is identified by a 16-digit bank account, an ORAM would require a prohibitive capacity of 10^{16} elements. Adopting tree structures on the ORAM makes the memory usage practical, but has a large complexity, e.g. $\Theta(|V|\log^2|V|\log^2\log|V|)$ [11]. In contrast, our method offers a low complexity while supporting a wide domain.

Hash bucketing may help build ChaosTable but incur inefficiency. Let us consider hashing the vertices into L buckets, each containing up to K vertices. Based on the number of allocated positions for each bucket, we may think of the following approaches, both with possible defects.

- If we allocate K positions for each bucket, the performance of each iteration will degrade. This is because the communication of each iteration is proportional to the result range $L \cdot K$, and we need $L = \Theta\left(\frac{|V|^{1+1/K}}{\varepsilon^{1/K} \cdot K}\right)$ to guarantee a failure probability of ε, which implies $L \cdot K = \Theta\left(\frac{|V|^{1+1/K}}{\varepsilon^{1/K}}\right)$, i.e. larger K results in better iteration performance. Thus, the optimal scenario is when K is sufficiently large, which means that we should not perform bucketing at all.
- If we allocate only the required number of positions for each bucket (based on the actual vertices in this bucket), the privacy or/and efficiency could suffer. A trivial implementation leaks the information about the number of vertices in each bucket, while fixing this privacy concern introduces additional overhead such as extra scanning passes and becomes less efficient. See Appendix C for details.

In conclusion, alternative methods do not help to develop more efficient implementations for building ChaosTable.

6 Evaluation

We present a variety of results to evaluate the performance of GraphAce. The focus is mainly on the comparison with GraphSC [43], since it is the state-of-the-art two-party graph analysis framework. Note that the performance results cannot be directly compared with other recent works due to different settings and/or tasks.[6] We evaluated the effects of various average degrees $|E|/|V|$. MP-SPDZ [27] provides a shortest-path algorithm, which is also evaluated and compared.

Table 2: Graphs used in experiments

Graph	$\lvert V_0\rvert$, $\lvert V_1\rvert$	$\lvert V\rvert$	$\lvert E\rvert$
Scale 14	128	192	16,192
Scale 17	1,024	1,536	129,536
Scale 20	8,192	12,288	1,036,288
Scale 24	131,072	196,608	16,580,608
Scale 27	1,048,576	1,572,864	132,644,864

6.1 Experimental Setup

The experiments are carried out using three different network conditions, five different graph applications, on five graphs.

Environments. To evaluate the speed under varying network conditions while utilizing identical computational resources, we execute the two parties on a single server, employing management software to introduce network latency and restrict the bandwidth. Each party can use 40 cores or 80 threads of an Intel Xeon 8380 processor, as well as 384 GB of DDR4-3200 memory. We consider three network configurations. (1) The single datacenter network (1-DC) with 1000 Mbps bandwidth and 2 ms latency emulates communication in a local network. This primarily serves as a reference and is practically unlikely to achieve security. (2) The inter-datacenter network (Inter-DC) with 100 Mbps bandwidth and 20 ms latency is appropriate for situations where parties are in the same country but not the same datacenter or city. (3) The Inter-Continent network, with 10 Mbps bandwidth and 200 ms latency, models the requirements over international or even intercontinental distances, where bandwidth is highly restricted, such as over an international dedicated line.

Graphs. The graphs used in the experiments are shown in Table 2. We intentionally configure the sum of the number of vertices and edges, $|V|+|E|$, to be a power of 2 to enhance the speed performance of GraphSC. Each graph is generated randomly, as the content does not affect the execution time of the secure solutions. For every graph, we assume $|E_0| = |E_1|$.

Applications. The following five graph applications are used:

- PageRank (PR) [45]. For each $v \in V_b$, P_b will learn the PageRank result of v in the whole graph.
- Sparse matrix-vector multiplication (SpMV). Each party has some non-zero values of $\mathbf{A}$ and $\mathbf{x}$, and learns the corresponding elements of $\mathbf{y}$, where $\mathbf{y} = \mathbf{A}^T\mathbf{x}$.

[6] For example, Araki et al. [2] (their 3-party honest majority vs. our 2-party dishonest majority), GraphOS [11] (their graph database with trusted processors vs. our graph analysis without special hardware), and Mazloom et al. [40] (their 4-party with differential privacy vs. our 2-party with no leakage other than public information). This applies similarly to other solutions.

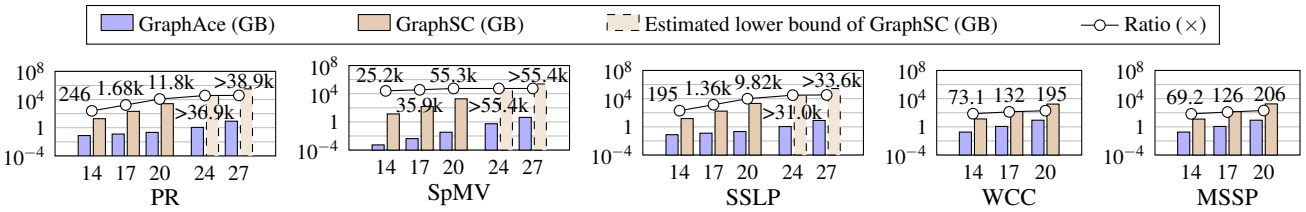


Figure 6: Communication amount per iteration of GraphAce and GraphSC (X-axis denotes graph scale)

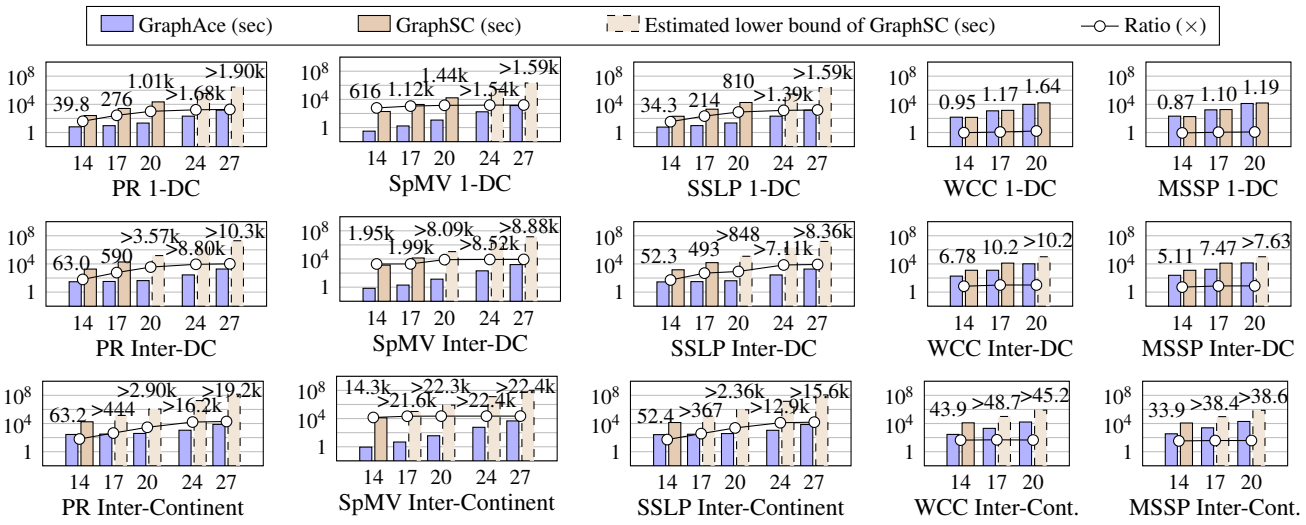


Figure 7: Time per iteration of GraphAce and GraphSC (X-axis denotes graph scale)

- Semi-supervised label propagation (SSLP) [36]. The training set combines the sets of both parties. If a vertex is in both sets, the averaged training feature is used. For $v \in V_b$, P_b learns the output features of v. As each vertex contains a vector of values, all reported results are per dimension.
- Weakly connected components (WCC). For $u, v \in V_b$, P_b learns whether they are connected on the whole graph.
- Multiple-source shortest path (MSSP). The set of starting vertex combines the sets from both parties. For each $v \in V_b$, P_b will learn the length of the shortest path from any starting vertex to v. The path is not revealed due to privacy.

We mainly adopt the Paillier [46] and the TFHE [13] schemes, practically satisfying the assumptions in Section 3.2 (see Appendix D for details). It is possible to implement PR, SpMV, and SSLP with Paillier, as no ciphertext multiplication is required, while WCC and MSSP have to use FHE.

6.2 Performance Comparison with GraphSC

To evaluate the performance of GraphAce and GraphSC, we performed experiments on the five applications using the five graphs, recording the execution time for the three network setups and the network traffic, averaged from three iterations. For any case where GraphSC would need more than 24 hours, we do not actually run it, but estimate its performance based on linear extrapolation of data from smaller cases. This is advantageous for GraphSC, as it does not account for the growth of the $\Theta(\log^2(|V|+|E|))$ factor in its complexity. For example, GraphSC running PR on scale 17 needs approximately 10.2× longer time than on scale 14 for Inter-DC, but the graph is only 8× larger.

Communication. Figure 6 shows the communication per iteration of GraphAce and GraphSC for various applications and graphs. The bars with dashed borders represent estimated lower bounds, and the corresponding ratios become minimal estimates. The y-axis including the ratio is on the log scale.

GraphAce better saves communication for larger graphs as its $\Theta(|V|)$ complexity is better than $\Theta((|V|+|E|)\log^2(|V|+|E|))$ of GraphSC. The saving ratio is even better when PHE is applicable, as the ciphertext to send is smaller, reducing the constant factor behind $\Theta(|V|)$. For the million-edge graph (scale 20), GraphAce can save 99.99% communication for PR, SpMV, and SSLP, or over 99% for WCC and MSSP.

Time. Figure 7 shows the results of execution time for different network configurations. Similarly, GraphAce offers better improvements on larger graphs due to lower complexity and for PHE cases due to cheaper operations. The ratio also increases with more demanding network conditions, due to the significant communication savings of GraphAce. GraphAce might be slower, but only for a *combination* of a small graph

Table 3: Scale-20 graphs of different average degrees $|E|/|V|$

Graph	$\|V_0\|, \|V_1\|$	$\|V\|$	$\|E\|$	$\|E\|/\|V\|$
I	8,192	12,288	1,036,288	84.33
II	16,384	24,576	1,024,000	41.67
III	32,768	49,152	999,424	20.33
IV	65,536	98,304	950,272	9.67
V	131,072	196,608	851,968	4.33

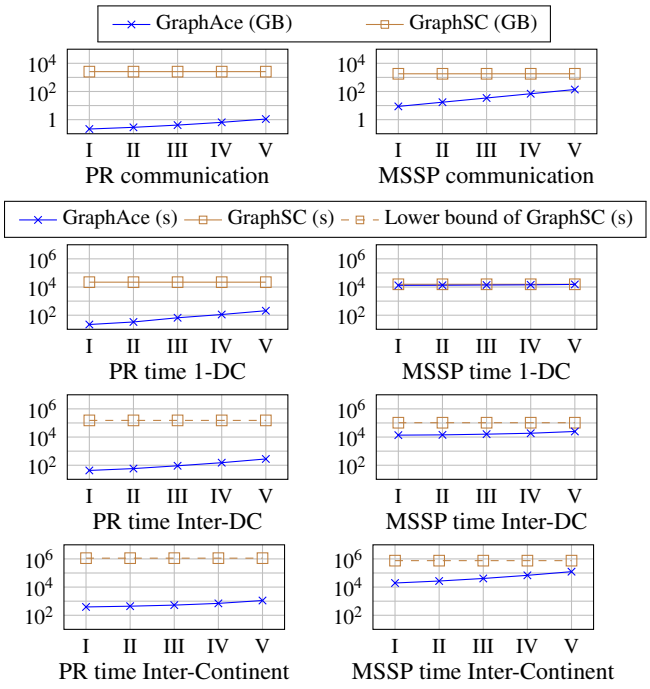


Figure 8: Performance per iter. on graphs of different $|E|/|V|$

(scale 14), an exessive network bandwidth (1-DC), and the need for an FHE scheme (WCC or MSSP). Even in such a worst case, GraphAce is at most 13% slower than GraphSC.

For the million-edge graph, GraphAce is expected to be about 1000× faster than GraphSC on PR, SpMV, and SSLP, while the speedup for WCC and MSSP depends mainly on the network, up to tens of times. GraphAce can process graph scale 27 ($|V| = 1.5 \times 10^6, |E| = 1.32 \times 10^8$), 128× larger than the one reported by GraphSC, even when the network is highly limited. GraphAce only needs 2.1 hours to perform a PR iteration on scale 27 for Inter-Continent, while even the optimistic estimate of GraphSC would require over 4.5 years. In this case, GraphAce is more than 19200× faster than GraphSC.

6.3 Influences of Average Degree

According to complexity analysis, the amount of communication depends only on $|V|$ for GraphAce, but on $|V|+|E|$ for GraphSC. Thus, it is necessary to perform experiments on graphs with different values of $|E|/|V|$, i.e. the average degree. As Table 3 shows, we choose five graphs with the same $|V|+|E| = 2^{20}$ but different average degrees, ranging from 4.33 to 84.33, reflecting the value of most real-world graphs [8, 9, 32]. The performance of GraphSC remains the same for all graphs, but that of GraphAce differs. To avoid displaying the results of all combinations of applications and graphs, we select two applications, PR and MSSP, representing the conditions of PHE and FHE, respectively.

Figure 8 shows the result of the communication and the time per iteration. Although the performance of GraphAce decreases for larger $|V|$, it still has a significant advantage over GraphSC even if the average degree is only 4.33. The impact on MSSP's execution time is minor (especially for 1-DC and Inter-DC), since FHE operations in the local message passing (LMP) with complexity $\Theta(|E|)$ are more expensive than PHE, and raising $|V|$ does not augment the cost of this part.

In summary, GraphAce can consistently surpass GraphSC on graphs of different average degrees.

6.4 Comparing Shortest Path of MP-SPDZ

MP-SPDZ [27] provides a shortest-path algorithm based on an oblivious heap. For two semi-honest parties, MP-SPDZ needs 107 hours to complete the execution on graph scale 14 under 1-DC, while GraphAce can run 1610 iterations given the same amount of time, which far exceeds the practical needs of a shortest-path algorithm. The amount of communication of MP-SPDZ is 3.31 TB, equivalent to 18780 iterations of GraphAce. Thus, GraphAce has a substantial advantage over MP-SPDZ. We choose not to test it on larger graphs or other network conditions because of its extended execution time.

7 Conclusion

We present GraphAce, an efficient secure two-party graph analysis framework. With a novel ChaosTable data structure, awareness of graph data location and the use of the mixed primitives system, GraphAce achieves a remarkable communication complexity of $\Theta(|V|)$ per iteration, in contrast to $\Omega(|V|+|E|)$ of previous work. Our evaluation shows that GraphAce outperforms GraphSC by up to tens of thousands of times in speed and communication. It can also process a graph with over 1 million vertices and 132 million edges within a reasonable time, which is of 128× larger scale than previously reported. We leave the cases where the two parties have largely imbalanced graph sizes for future work.

Acknowledgments

The authors thank the reviewers and the shepherd for their valuable feedback. This work was supported by Ant Group Research Intern Program. This work was partially supported by NSFC U20B2044. The corresponding authors are Kun Chen and Wenguang Chen.

Ethics Considerations

This study is in the area of privacy-preserving computation with the principal goal of protecting human privacy.

The evaluation process poses no ethical issues, as it utilizes synthetic data sets instead of real human data (which suffices, as the performance of the solutions being compared is determined by the dataset size, independent of the contents).

Following publication, no real-world harms or human rights violations are anticipated, since what we do is to offer an additional approach to perform secure two-party graph analysis. Our framework significantly reduces information leakage compared to insecure two-party solutions, thus protecting human rights. Moreover, compared to existing secure two-party solutions such as GraphSC [43], our method offers greater efficiency while maintaining a similar level of security.

Ultimately, this research is expected to not result in adverse outcomes from both the research and publication processes.

Open Science

The artifact is available at https://zenodo.org/records/14720419, including:

- Source code of our implementation of GraphAce.
- Documentation on how to compile the source code and execute the program, including the dependency specifications (e.g., compilers, libraries).
- Scripts to run the experiments.
- Data sets for the experiments (actually generated by the program given the data size, since random data sets are sufficient for evaluation).

References

[1] Mark Abspoel, Daniel Escudero, and Nikolaj Volgushev. Secure training of decision trees with continuous attributes. *Proceedings on Privacy Enhancing Technologies (PoPETs)*, 2021.

[2] Toshinori Araki, Jun Furukawa, Kazuma Ohara, Benny Pinkas, Hanan Rosemarin, and Hikaru Tsuchida. Secure graph analysis at scale. In *Proceedings of ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2021.

[3] Sofiane Azogagh, Zelma Aubin Birba, Sébastien Gambs, and Marc-Olivier Killijian. Crypto'graph: Leveraging privacy-preserving distributed link prediction for robust graph learning. *arXiv preprint arXiv:2309.10890*, 2023.

[4] Kenneth E Batcher. Sorting networks and their applications. In *Proceedings of the April 30–May 2, 1968, spring joint computer conference*, pages 307–314, 1968.

[5] Donald Beaver. Efficient multiparty protocols using circuit randomization. In *Advances in Cryptology—CRYPTO'91: Proceedings 11*, pages 420–432. Springer, 1992.

[6] Marina Blanton, Aaron Steele, and Mehrdad Alisagari. Data-oblivious graph algorithms for secure computation and outsourcing. In *Proceedings of the ACM SIGSAC symposium on Information, computer and communications security (CCS)*, 2013.

[7] Mark Blunk, Paul Bunn, Samuel Dittmer, Steve Lu, and Rafail Ostrovsky. Secure merge in linear time and o (log log n) rounds. *Cryptology ePrint Archive*, 2022.

[8] Paolo Boldi, Marco Rosa, Massimo Santini, and Sebastiano Vigna. Layered label propagation: A multiresolution coordinate-free ordering for compressing social networks. In Sadagopan Srinivasan, Krithi Ramamritham, Arun Kumar, M. P. Ravindra, Elisa Bertino, and Ravi Kumar, editors, *Proceedings of the 20th international conference on World Wide Web*, pages 587–596. ACM Press, 2011.

[9] Paolo Boldi and Sebastiano Vigna. The WebGraph framework I: Compression techniques. In *Proc. of the Thirteenth International World Wide Web Conference (WWW 2004)*, pages 595–601, Manhattan, USA, 2004. ACM Press.

[10] Beyza Bozdemir, Sébastien Canard, Orhan Ermis, Helen Möllering, Melek Önen, and Thomas Schneider. Privacy-preserving density-based clustering. In *Proceedings of the 2021 ACM Asia Conference on Computer and Communications Security (Asia CCS)*, 2021.

[11] Javad Ghareh Chamani, Ioannis Demertzis, Dimitrios Papadopoulos, Charalampos Papamanthou, and Rasool Jalili. Graphos: Towards oblivious graph processing. *Cryptology ePrint Archive*, 2024.

[12] Rong Chen, Jiaxin Shi, Yanzhe Chen, Binyu Zang, Haibing Guan, and Haibo Chen. Powerlyra: Differentiated graph computation and partitioning on skewed graphs. *ACM Transactions on Parallel Computing (TOPC)*, 2019.

[13] Ilaria Chillotti, Nicolas Gama, Mariya Georgieva, and Malika Izabachène. TFHE: fast fully homomorphic encryption over the torus. *Journal of Cryptology*, 2020.

[14] Kelong Cong, Debajyoti Das, Jeongeun Park, and Hilder VL Pereira. Sortinghat: Efficient private decision tree evaluation via homomorphic encryption and transciphering. In *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2022.

[15] Anders Peter Kragh Dalskov, Daniel Esteban Escudero Ospina, and Marcel Keller. Secure evaluation of quantized neural networks. *Proceedings on Privacy Enhancing Technologies (PoPETs)*, 2020, 2020.

[16] Ivan Damgård, Valerio Pastro, Nigel Smart, and Sarah Zakarias. Multiparty computation from somewhat homomorphic encryption. In *Annual Cryptology Conference*. Springer, 2012.

[17] Daniel Demmler, Thomas Schneider, and Michael Zohner. ABY-a framework for efficient mixed-protocol secure two-party computation. In *NDSS*, 2015.

[18] Minxin Du, Peipei Jiang, Qian Wang, Sherman SM Chow, and Lingchen Zhao. Shielding graph for exact analytics with SGX. *IEEE Transactions on Dependable and Secure Computing*, 2023.

[19] Wenjing Fang, Derun Zhao, Jin Tan, Chaochao Chen, Chaofan Yu, Li Wang, Lei Wang, Jun Zhou, and Benyu Zhang. Large-scale secure XGB for vertical federated learning. In *Proceedings of the ACM International Conference on Information & Knowledge Management (CIKM)*, 2021.

[20] Oded Goldreich. *Foundations of cryptography: volume 2, basic applications*. Cambridge university press, 2009.

[21] Oded Goldreich, Silvio Micali, and Avi Wigderson. How to play any mental game, or a completeness theorem for protocols with honest majority. In *Providing Sound Foundations for Cryptography: On the Work of Shafi Goldwasser and Silvio Micali*. 2019.

[22] Joseph E Gonzalez, Yucheng Low, Haijie Gu, Danny Bickson, and Carlos Guestrin. PowerGraph: Distributed Graph-Parallel computation on natural graphs. In *USENIX symposium on operating systems design and implementation (OSDI)*, pages 17–30, 2012.

[23] Yuke Hu, Wei Liang, Ruofan Wu, Kai Xiao, Weiqiang Wang, Xiaochen Li, Jinfei Liu, and Zhan Qin. Quantifying and defending against privacy threats on federated knowledge graph embedding. In *Proceedings of the ACM Web Conference 2023*, pages 2306–2317, 2023.

[24] Zhicong Huang, Wen-jie Lu, Cheng Hong, and Jiansheng Ding. Cheetah: Lean and fast secure two-party deep neural network inference. In *USENIX Security Symposium (USENIX Security)*, pages 809–826, 2022.

[25] Jacob Imola, Takao Murakami, and Kamalika Chaudhuri. Locally differentially private analysis of graph statistics. In *USENIX security symposium (USENIX Security)*, 2021.

[26] Jacob Imola, Takao Murakami, and Kamalika Chaudhuri. Communication-efficient triangle counting under local differential privacy. In *USENIX security symposium (USENIX Security)*, pages 537–554, 2022.

[27] Marcel Keller. MP-SPDZ: A versatile framework for multi-party computation. In *Proceedings of the ACM SIGSAC conference on computer and communications security (CCS)*, pages 1575–1590, 2020.

[28] Kamil Kluczniak. Circuit privacy for FHEW/TFHE-style fully homomorphic encryption in practice. *Cryptology ePrint Archive*, 2022.

[29] Nishat Koti, Varsha Bhat Kukkala, Arpita Patra, and Bhavish Raj Gopal. Entrada to secure graph convolutional networks. *Cryptology ePrint Archive*, 2023.

[30] Amy N Langville and Carl D Meyer. Deeper inside pagerank. *Internet Mathematics*, 1(3):335–380, 2004.

[31] Ryan Lehmkuhl, Pratyush Mishra, Akshayaram Srinivasan, and Raluca Ada Popa. Muse: Secure inference resilient to malicious clients. In *USENIX Security Symposium (USENIX Security)*, 2021.

[32] Jure Leskovec and Andrej Krevl. SNAP Datasets: Stanford large network dataset collection. http://snap.stanford.edu/data, June 2014.

[33] Kai Li, Xin Yuan, Jingjing Zheng, Wei Ni, Falko Dressler, and Abbas Jamalipour. Leverage variational graph representation for model poisoning on federated learning. *arXiv preprint arXiv:2404.15042*, 2024.

[34] Kai Li, Xin Yuan, Jingjing Zheng, Wei Ni, and Mohsen Guizani. Exploring adversarial graph autoencoders to manipulate federated learning in the internet of things. In *2023 International Wireless Communications and Mobile Computing (IWCMC)*, pages 898–903. IEEE, 2023.

[35] Kai Li, Jingjing Zheng, Xin Yuan, Wei Ni, Ozgur B Akan, and H Vincent Poor. Data-agnostic model poisoning against federated learning: A graph autoencoder approach. *IEEE Transactions on Information Forensics and Security*, 2024.

[36] Dong Liu, Hong-Yu Bai, Hui-Jia Li, and Wen-Jun Wang. Semi-supervised community detection using label propagation. *International Journal of Modern Physics B*, 2014.

[37] Yuhan Liu, Tianhao Wang, Yixuan Liu, Hong Chen, and Cuiping Li. Edge-protected triangle count estimation under relationship local differential privacy. *IEEE Transactions on Knowledge and Data Engineering (TKDE)*, 2024.

[38] Wen-jie Lu, Zhicong Huang, Qizhi Zhang, Yuchen Wang, and Cheng Hong. Squirrel: A scalable secure two-party computation framework for training gradient boosting decision tree. *Cryptology ePrint Archive*, 2023.

[39] Sahar Mazloom and S Dov Gordon. Secure computation with differentially private access patterns. In *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2018.

[40] Sahar Mazloom, Phi Hung Le, Samuel Ranellucci, and S Dov Gordon. Secure parallel computation on national scale volumes of data. In *USENIX Security Symposium (USENIX Security)*, 2020.

[41] Xianrui Meng, Dimitrios Papadopoulos, Alina Oprea, and Nikos Triandopoulos. Private hierarchical clustering and efficient approximation. In *Proceedings of the 2021 on Cloud Computing Security Workshop*, 2021.

[42] Pratyush Mishra, Ryan Lehmkuhl, Akshayaram Srinivasan, Wenting Zheng, and Raluca Ada Popa. Delphi: A cryptographic inference system for neural networks. In *Proceedings of the Workshop on Privacy-Preserving Machine Learning in Practice*, 2020.

[43] Kartik Nayak, Xiao Shaun Wang, Stratis Ioannidis, Udi Weinsberg, Nina Taft, and Elaine Shi. GraphSC: Parallel secure computation made easy. In *IEEE Symposium on Security and Privacy (S&P)*. IEEE, 2015.

[44] Donald Nguyen, Andrew Lenharth, and Keshav Pingali. A lightweight infrastructure for graph analytics. In *Proceedings of the ACM symposium on operating systems principles (SOSP)*, pages 456–471, 2013.

[45] Lawrence Page, Sergey Brin, Rajeev Motwani, Terry Winograd, et al. The pagerank citation ranking: Bringing order to the web. 1999.

[46] Pascal Paillier. Public-key cryptosystems based on composite degree residuosity classes. In *International conference on the theory and applications of cryptographic techniques*. Springer, 1999.

[47] Qi Pang, Jinhao Zhu, Helen Möllering, Wenting Zheng, and Thomas Schneider. Bolt: Privacy-preserving, accurate and efficient inference for transformers. In *IEEE Symposium on Security and Privacy (S&P)*. IEEE Computer Society, 2024.

[48] Xiafei Qiu, Wubin Cen, Zhengping Qian, You Peng, Ying Zhang, Xuemin Lin, and Jingren Zhou. Real-time constrained cycle detection in large dynamic graphs. *Proceedings of the VLDB Endowment*, 2018.

[49] Deevashwer Rathee, Mayank Rathee, Nishant Kumar, Nishanth Chandran, Divya Gupta, Aseem Rastogi, and Rahul Sharma. Cryptflow2: Practical 2-party secure inference. In *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2020.

[50] Amitabha Roy, Laurent Bindschaedler, Jasmina Malicevic, and Willy Zwaenepoel. Chaos: Scale-out graph processing from secondary storage. In *Proceedings of the Symposium on Operating Systems Principles (SOSP)*, 2015.

[51] Amitabha Roy, Ivo Mihailovic, and Willy Zwaenepoel. X-stream: Edge-centric graph processing using streaming partitions. In *Proceedings of the ACM Symposium on Operating Systems Principles (SOSP)*, pages 472–488, 2013.

[52] Sina Sajadmanesh, Ali Shahin Shamsabadi, Aurélien Bellet, and Daniel Gatica-Perez. GAP: Differentially private graph neural networks with aggregation perturbation. In *32nd USENIX Security Symposium (USENIX Security 23)*, pages 3223–3240, 2023.

[53] Alex Sangers, Maran van Heesch, Thomas Attema, Thijs Veugen, Mark Wiggerman, Jan Veldsink, Oscar Bloemen, and Daniël Worm. Secure multiparty pagerank algorithm for collaborative fraud detection. In *Financial Cryptography and Data Security (FC)*. Springer, 2019.

[54] Baoling Shan, Xin Yuan, Wei Ni, Xin Wang, Ren Ping Liu, and Eryk Dutkiewicz. Preserving the privacy of latent information for graph-structured data. *IEEE Transactions on Information Forensics and Security*, 2023.

[55] Julian Shun and Guy E Blelloch. Ligra: a lightweight graph processing framework for shared memory. In *Proceedings of the ACM SIGPLAN symposium on Principles and practice of parallel programming*, pages 135–146, 2013.

[56] Haipei Sun, Xiaokui Xiao, Issa Khalil, Yin Yang, Zhan Qin, Hui Wang, and Ting Yu. Analyzing subgraph statistics from extended local views with decentralized differential privacy. In *Proceedings of the ACM SIGSAC conference on computer and communications security (CCS)*, 2019.

[57] Cheng-Long Wang, Mengdi Huai, and Di Wang. Inductive graph unlearning. In *32nd USENIX Security Symposium (USENIX Security 23)*, pages 3205–3222, 2023.

[58] Songlei Wang, Yifeng Zheng, and Xiaohua Jia. Graphguard: Private time-constrained pattern detection over streaming graphs in the cloud.

[59] Zhigang Wang, Yu Gu, Yubin Bao, Ge Yu, and Jeffrey Xu Yu. Hybrid pulling/pushing for i/o-efficient distributed and iterative graph computing. In *Proceedings of the International Conference on Management of Data (SIGMOD)*, pages 479–494, 2016.

[60] Kun Wu, Jie Shen, Yue Ning, Ting Wang, and Wendy Hui Wang. Certified edge unlearning for graph neural networks. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pages 2606–2617, 2023.

[61] Yuncheng Wu, Shaofeng Cai, Xiaokui Xiao, Gang Chen, and Beng Chin Ooi. Privacy preserving vertical federated learning for tree-based models. *arXiv preprint arXiv:2008.06170*, 2020.

[62] Zhaohan Xi, Tianyu Du, Changjiang Li, Ren Pang, Shouling Ji, Xiapu Luo, Xusheng Xiao, Fenglong Ma, and Ting Wang. On the security risks of knowledge graph reasoning. In *32nd USENIX Security Symposium (USENIX Security 23)*, pages 3259–3276, 2023.

[63] Siyuan Xia, Beizhen Chang, Karl Knopf, Yihan He, Yuchao Tao, and Xi He. Dpgraph: A benchmark platform for differentially private graph analysis. In *Proceedings of the International Conference on Management of Data (SIGMOD)*, 2021.

[64] Zihang Xiang, Tianhao Wang, and Di Wang. Preserving node-level privacy in graph neural networks. *arXiv preprint arXiv:2311.06888*, 2023.

[65] Zhibo Xu, Shangqi Lai, Xiaoning Liu, Alsharif Abuadbba, Xingliang Yuan, and Xun Yi. OblivGNN: Oblivious inference on transductive and inductive graph neural network.

[66] Andrew Chi-Chih Yao. How to generate and exchange secrets. In *Annual Symposium on Foundations of Computer Science (FOCS)*. IEEE, 1986.

[67] Quan Yuan, Zhikun Zhang, Linkang Du, Min Chen, Peng Cheng, and Mingyang Sun. PrivGraph: Differentially private graph data publication by exploiting community information. In *32nd USENIX Security Symposium (USENIX Security 23)*, pages 3241–3258, 2023.

[68] Pu Zhao, Jia Yu, Hanlin Zhang, Zhan Qin, and Cong Wang. How to securely outsource finding the min-cut of undirected edge-weighted graphs. *IEEE Transactions on Information Forensics and Security*, 15:315–328, 2019.

[69] Xiaochen Zhu, Vincent YF Tan, and Xiaokui Xiao. Blink: Link local differential privacy in graph neural networks via bayesian estimation. In *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2023.

[70] Xiaowei Zhu, Wenguang Chen, Weimin Zheng, and Xiaosong Ma. Gemini: A Computation-Centric distributed graph processing system. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2016.

[71] Xiaowei Zhu, Wentao Han, and Wenguang Chen. GridGraph: Large-Scale graph processing on a single machine using 2-level hierarchical partitioning. In *USENIX Annual Technical Conference (ATC)*, pages 375–386, 2015.

A Security Analysis

GraphAce is capable of securely performing two-party graph analysis (Section 3.2) in the presence of semi-honest adversaries. Since the output of each party is well defined (deterministic) by public information and input, and the correctness has been demonstrated, we only need to show the privacy to complete the security analysis. We adopt a simulation-based approach as follows. Without loss of generity, we assume that P_0 is compromised. Because of the correctness, we can adopt a proved result (Section 7.2.2 in [20]) that we only need to give a simulator S for P_0 such that $\{S(In_0, Out_0)\}_{In_0,In_1} \overset{c}{\equiv} \{\text{View}_0^{\text{GraphAce}}(In_0, In_1)\}_{In_0,In_1}$, where $\text{View}_0^{\text{GraphAce}}(In_0, In_1)$ denotes the view (including the input, the received messages and the consumed random bits) of P_0 during an execution with P_0 inputting In_0 and P_1 inputting In_1. Notice that external protocols have strong security properties:

Theorem 4. *To simulate received messages and generated random numbers, calling the simulators of the following protocols does not need to provide the input or the output of an actual protocol execution: (1) evaluating any public circuit on arithmetic shares (including possible conversion to/from binary shares); (2)* SecureMerge, RevertSecureMerge, *and* GenerateRandomPermutation*; (3) converting between arithmetic shares and HE ciphertext (*A2H, A2H_b *and* H2A_b*); (4)* RevealHE_b *(only when simulating* P_{1-b}*).*

To simulate P_b *in* RevealHE_b*, the simulator only needs the actual output x (without the input* $[\![x]\!]_b^H$*).*

Security demonstrations of (1) can be found in existing studies such as [17]. For (2), consult the corresponding secure merge realization, e.g. odd-even merge can be implemented purely with circuits and satisfies (1). For protocols regarding homomorphic encryption, refer to Appendix D.

Theorem 5. *The overall protocol of GraphAce (Algorithm 7) securely reveals the required output to the parties, given the input and public information, as defined in Section 3.2.*

Proof (outline). We construct a simulator S for P_0. First, S directly outputs In_0 as a part of the view.

We observe that all protocols in Section 4 themselves do not directly generate random numbers or receive any messages, but only in the subprotocols. Thus, S only needs to call the simulators of external subprotocols. Due to Theorem 4, S does not need to provide any parameter for most calls to external simulators (i.e., everything except $\mathsf{RevealHE}_0$).

For the $\mathsf{RevealHE}_0$ calls in BuildChaosTable, S uniformly randomly samples $N/2$ different numbers from $[N]$, and uses them as parameters to call the simulator of $\mathsf{RevealHE}_0$ for $N/2$ times. Theorem 2 and the semantic security of the HE scheme together guarantee that such a distribution is computationally indistinguishable from that of the actual views.

For the $\mathsf{RevealHE}_0$ calls in RevealResults, S gets the corresponding results from Out_0 and uses them (and the $N/2-|V_b|$ "garbage" values of 0_I) to call the simulator of $\mathsf{RevealHE}_0$.

Thus, S is constructed, establishing overall security. □

B Discussion of Public Information

Although GraphAce and GraphSC have the same semi-honest 2-party (dishonest majority) setting, the public information of GraphAce (P_b learns $\lceil \log_2 |V_{1-b}| \rceil$) and that of GraphSC (both parties learn $|V|+|E|$) is slightly different. We believe that such a difference does not make the security level of any framework stronger or weaker than the other's. In GraphSC, each party knows the total number of vertices and edges $|V|+|E|$, thus (at least) obtaining the upper bound of $|V|$ and $|E|$. In GraphAce, P_b knows the value $\lceil \log_2 |V_{1-b}| \rceil$, which means that $|V_{1-b}|$ lies in a certain range. Although the range of the number of vertex seems a bit more accurate, P_b learns "completely nothing" about $|E_{1-b}|$ from public information, on the other hand. Consequently, neither framework requires the disclosure of "strictly more information" compared to the other. Additionally, in both frameworks, the public information pertains to the length of the array involved in the multi-party computation, and secure algorithms on the array can only function when such information is revealed. Thus, we consider the frameworks to have a similar security level.

Also, GraphSC does not provide implementations to convert the input vertices and edges of each party to the desired garbled array of tuples, and the above discussions do not account for potential needs of public information during such preprocessing. For example, possible preprocessing implementations may require P_b to learn $|V_{1-b}|$ or $|V_{1-b}|+|E_{1-b}|$.

C Details of Bucketing to Build ChaosTable

Algorithm 8 shows our reference implementation of hash bucketing as an alternative method of building ChaosTable. We assume that the parameters are predefined, so that each party group its vertices into L buckets, each with at most K vertices, and the parties agree with a hash function $h(v)$ that maps a vertices to $[L]$ (e.g. salted SHA-256 modulo L).

Algorithm 8 BuildChaosTable (with hash bucketing)

Requires: Each P_b holds V_b. Public parameters L, K, and a hash function h that maps a vertex to $[L]$
Ensures: Each P_b obtains $p(v)$ for $v \in V_b$ (see below).

// Part I: put vertices into buckets
Each P_b prepares a two-dimensional array $A_b[L][K]$:
  $A_b \leftarrow$ an array of L empty arrays
  for all $v \in V_b$ **do**
    Append v to $A_b[h(v)]$
  for all $l \in [L]$ **do**
    if $A_b[l]$ has more than K elements **then**
      throw protocol fails
    Sort $A_b[l]$ in ascending order
    Resize $A_b[l]$ to be K elements, filling with the last (largest) element of $A_b[l]$
Each P_b shares A_b, resulting $[\![A_0]\!]$ and $[\![A_1]\!]$

// Part II: allocate (insecure) positions into $B_b[][]$
$[\![s]\!] \leftarrow [\![0]\!]$
for all $b \in \{0,1\}$ **do**
  $[\![B_b]\!] \leftarrow$ an array of L arrays
for all $l \in [L]$ **do**
  $[\![A]\!], [\![\text{trace}]\!] \leftarrow \mathsf{SecureMerge}([\![A_0[l]]\!], [\![A_1[l]]\!])$
  $[\![B]\!] \leftarrow$ an array of $2 \cdot K$ elements
  for all $k \in [2 \cdot K]$ **do**
    if $k = 1$ **then**
      $[\![B[k]]\!] \leftarrow \mathsf{Add}([\![s]\!], [\![1]\!])$
    else
      $[\![B[k]]\!] \leftarrow \mathsf{NotEqual}([\![A[k]]\!], [\![A[k-1]]\!])$
  $[\![B]\!] \leftarrow \mathsf{PrefixSum}([\![B]\!])$
  $[\![s]\!] \leftarrow [\![B[2 \cdot K]]\!]$
  $[\![B_0[l]]\!], [\![B_1[l]]\!] \leftarrow \mathsf{RevertSecureMerge}([\![B]\!], [\![\text{trace}]\!])$

// Part III: transform insecure positions into secure ones
$N \leftarrow 2^{1+\max(\lceil \log_2 |V_0| \rceil, \lceil \log_2 |V_1| \rceil)}$
$[\![R]\!] \leftarrow \mathsf{GenerateRandomPermutation}([\![[1,2,\ldots,N]]\!])$
for all $b \in \{0,1\}$ **do**
  $[\![C_1]\!] \leftarrow$ an array of $L \cdot K$ pairs (before, after)
  for all $l \in [L]$ **do**
    for all $k \in [K]$ **do**
      $[\![C_1[l \cdot K + k].\text{before}]\!] \leftarrow [\![B_b[l][k]]\!]$
      $[\![C_1[l \cdot K + k].\text{after}]\!] \leftarrow [\![2^W - 1]\!]$
  $[\![C_2]\!] \leftarrow$ an array of N pairs (before, after)
  for all $i \in [N]$ **do**
    $[\![C_2[i].\text{before}]\!] \leftarrow [\![i]\!]$
    $[\![C_2[i].\text{after}]\!] \leftarrow [\![R[i]]\!]$
  $[\![C]\!], [\![\text{trace}]\!] \leftarrow \mathsf{RevertSecureMerge}([\![C_1]\!], [\![C_2]\!])$
  for all $i \in [L \cdot K + N - 1]$ **do**
    $[\![C[i+1].\text{after}]\!] \leftarrow \mathsf{IfThenElse}($
    $\mathsf{Equal}([\![C[i].\text{before}]\!], [\![C[i+1].\text{before}]\!]), [\![C[i].\text{after}]\!], [\![C[i+1].\text{after}]\!])$
  $[\![D]\!] \leftarrow$ array of $L \cdot K + N$ elements, each $[\![D[i]]\!] = [\![C[i].\text{after}]\!]$
  $[\![D_1]\!], [\![D_2]\!] \leftarrow \mathsf{RevertSecureMerge}([\![D]\!], [\![\text{trace}]\!])$
  for all $l \in [L]$ **do**
    for all $k \in [K]$ **do**
      Reveal $D_1[l \cdot K + k]$ to P_b
      P_b executes:
        $p(A_b[l][k]) \leftarrow D_1[l \cdot K + k]$

The protocol starts with local computation (Part I) where each party P_b prepares a two-dimensional array $A_b[L][K]$ representing the buckets. Each bucket is sorted to accelerate future secure computations, and is resized to K elements. Next, Part II allocate positions for each bucket and store them in a result array $B_b[][]$, corresponding to each vertex in $A_b[][]$. Since

we need to avoid allocating K positions comprehensively for each bucket (otherwise, the result would be suboptimal; see Section 5.2), we only allocate the required positions, corresponding to the number of vertices in each bucket. To count this, we use a secure merge and inspect whether the adjacent elements are equal or not. After Part II, (the corresponding plaintext of) array $B_b[][]$ contains a valid position for each vertex. However, we cannot reveal $B_b[][]$ to each P_b, because it would leak information about the number of vertices that P_{1-b} knows, for each bucket. Thus, Part III transforms the values according to a random array. We generate a random permutation and use $B_b[][]$ to access it, which can be realized in a secure-merge fashion. After the transformation, we can safely reveal the corresponding elements to the parties.

Compared to this alternative approach based on hash bucketing, our proposed method is simpler and more efficient (because of fewer invocations of secure merging).

D Details of Homomorphic Encryption

In Section 3.2, we assumed that the HE scheme satisfies semantic security (IND-CPA) and that it is possible to implement the conversion between SS and HE. Although the former is a basic requirement for an HE scheme to be secure, the latter may not be thoroughly discussed during the design of HE schemes. Algorithm 9 provide possible implementations of conversion between SS and HE, as well as the simulators satisfying Theorem 4. This implementation requires that the HE scheme satisfies the following:[7] (1) Homomorphic addition with plaintext. An operator $\boxplus$ is available to add a ciphertext and a plaintext. In mathematical terms, for a ciphertext c and a plaintext $x \in \mathbb{Z}_{2^W}$, it is guaranteed that $\mathsf{Decrypt}(\mathrm{sk}, c \boxplus x) = (\mathsf{Decrypt}(\mathrm{sk}, c) + x) \bmod 2^W$. (2) Circuit privacy. Information beyond the deciphered value cannot be inferred from a ciphertext when the secret key is known. In formal terms, the distributions of $f(\mathsf{Encrypt}(\mathrm{pk}, x_1), \ldots, \mathsf{Encrypt}(\mathrm{pk}, x_k))$ and $\mathsf{Encrypt}(\mathrm{pk}, f(x_1, \ldots, x_k))$ should be computationally indistinguishable for any function f and the input values $x_1, \ldots, x_k$.

We used Paillier and TFHE schemes in our evaluation processes. The Paillier cryptosystem satisfies circuit privacy because it supports a rerandomization operation that transforms a ciphertext to resemble a freshly encrypted value while keeping the corresponding plaintext secure, even without the knowledge of secret key. Although it does not support an exact homomorphic addition modulo 2^W (because its modulus is dependent on the key pair), we can let the modulus be close to 2^W with a negligible factor (e.g., 2^{-256}), so that we would not lose the precision of the computation and does not practically break the security requirements. TFHE naturally supports homomorphic addition with plaintext (modulo 2^W). Though the circuit privacy of the original TFHE scheme is not formally established, there have been no known practical exploits to effectively recover the circuit. In contrast, there are methods to ensure circuit privacy [28]. Therefore, we assume that TFHE can practically achieve circuit privacy in this work.

Based on the conversion from HE to SS, Algorithm 10 provide a possible implementation to securely reveal a HE ciphertext to a party P_b without information leakage, with the help of the other party P_{1-b}. The corresponding simulators are also given, satisfying Theorem 4.

[7] These requirements are not mandatory for an HE scheme to be used. A scheme may provide secure conversion between SS and HE in other ways.

Algorithm 9 Security of conversion between SS and HE

function $\mathsf{A2H}_b([\![x]\!])$
Output: P_b obtains a HE ciphertext $[\![x]\!]^H_b$
1: Party P_b executes: $[\![x]\!]^H_b \leftarrow \mathsf{Receive}() \boxplus [\![x]\!]_b$
2: Party P_{1-b} executes: $\mathsf{Send}(\mathsf{Encrypt}(\mathrm{pk}_{1-b}, [\![x]\!]_{1-b}))$

Input- and output-independent simulator of $\mathsf{A2H}_b$
3: For party P_b: ReceivedMessages $\leftarrow \mathsf{Encrypt}(\mathrm{pk}_{1-b}, \mathsf{Rand}() \bmod 2^W)$
// Indistinguishable with the actual message distribution; otherwise, the HE scheme does not satisfy IND-CPA
4: For party P_{1-b}: *// Output nothing*

function $\mathsf{A2H}([\![x]\!]_b)$
Output: a homomorphic ciphertext $[\![x]\!]^H_b$
5: Call $\mathsf{A2H}_0$ and $\mathsf{A2H}_1$

Input- and output-independent simulator of A2H
6: Call the simulator of $\mathsf{A2H}_0$ and $\mathsf{A2H}_1$

function $\mathsf{H2A}_b([\![x]\!]^H_b$ from $P_b)$
Output: an arithmetic share $[\![x]\!]$
7: Party P_b executes:
8: $\quad y \leftarrow \mathsf{Rand}() \bmod 2^W$
9: $\quad \mathsf{Send}([\![x]\!]^H_b \boxplus y)$
10: $\quad [\![x]\!]_b \leftarrow (-y) \bmod 2^W$
11: Party P_{1-b} executes:
12: $\quad [\![x]\!]_{1-b} \leftarrow \mathsf{Decrypt}(\mathrm{sk}_{1-b}, \mathsf{Receive}())$

Input- and output-independent simulator of $\mathsf{H2A}_b$
13: For party P_b: RandomBits $\leftarrow \mathsf{Rand}() \bmod 2^W$
14: For party P_{1-b}:
15: $\quad$ ReceivedMessages $\leftarrow \mathsf{Encrypt}(\mathrm{pk}_{1-b}, \mathsf{Rand}() \bmod 2^W)$
// Indistinguishable with the actual message distribution because (1) the decrypted value $[\![x]\!]_{1-b}$ is uniform over $\mathbb{Z}_{2^W}$, of which the randomness comes solely of the other party's ad hoc random value, and (2) the HE scheme preserves circuit privacy

Algorithm 10 Security of revealing HE ciphertext

function $\mathsf{RevealHE}_b([\![x]\!]^H_b$ from $P_b)$
Output: P_b obtains a plaintext x
1: Party P_b executes:
2: $\quad [\![x]\!]_b \leftarrow \mathsf{H2A}_b([\![x]\!]^H_b)$
3: $\quad x \leftarrow ([\![x]\!]_b + \mathsf{Receive}()) \bmod 2^W$
4: Party P_{1-b} executes:
5: $\quad [\![x]\!]_{1-b} \leftarrow \mathsf{H2A}_b()$
6: $\quad \mathsf{Send}([\![x]\!]_{1-b})$

Input-independent simulator of $\mathsf{RevealHE}_b(x)$ for P_b
// x is the actual return value of $\mathsf{RevealHE}_b$
7: $y \leftarrow \mathsf{Rand}() \bmod 2^W$
8: RandomBits $\leftarrow y$
9: ReceivedMessages $\leftarrow (x+y) \bmod 2^W$

Input- and output-independent simulator of $\mathsf{RevealHE}_b$ for P_{1-b}
10: Call the simulator of $\mathsf{H2A}_b$ for P_{1-b}